VIETNAM NATIONAL UNIVERSITY, HANOI

UNIVERSITY OF ENGINEERING AND TECHNOLOGY



Bui Minh Thang

DESIGN AND EVALUATION OF A CONTEXT-AWARE RESOURCE PROFILE SELECTION EXTENSION FOR JUPYTERHUB ON KUBERNETES

GRADUATION INTERNSHIP

Major: Computer Networks and Data Communications

HA NOI – 2026

VIETNAM NATIONAL UNIVERSITY, HANOI

UNIVERSITY OF ENGINEERING AND TECHNOLOGY

Bui Minh Thang

DESIGN AND EVALUATION OF A CONTEXT-AWARE RESOURCE PROFILE SELECTION EXTENSION FOR JUPYTERHUB ON KUBERNETES

GRADUATION INTERNSHIP

Major: Computer Networks and Data Communications

Supervisor: Dr. Hoang Xuan Tung

HA NOI – 2026

ABSTRACT

JupyterHub provides a flexible platform for interactive computing, but selecting an appropriate computational environment remains a manual configuration task. Users are typically required to understand available resource profiles, software environments, and notebook images before starting a session. As the number and diversity of available environments increase, manual selection can introduce unnecessary configuration effort and may result in unsuitable resource or software choices. This internship project investigates the use of workload intent and context to assist users in selecting an appropriate JupyterHub environment while preserving administrator-defined resource and deployment policies.

The problem addressed in this internship report is the selection of a suitable resource profile and notebook environment from a bounded, administrator-managed catalog based on a user's natural-language workload description. The objective is not to allow an automated model to generate arbitrary Kubernetes configurations, but to determine whether natural-language intent can be transformed into a structured representation, used to retrieve relevant approved environments, and subsequently filtered and ranked according to explicit technical constraints. A further objective is to establish a system architecture in which recommendation remains explainable, policy-bounded, reproducible, and subject to user confirmation before deployment.

The proposed approach, implemented as Intent-Spawner, combines structured intent extraction, hybrid information retrieval, deterministic constraint evaluation, and trusted catalog resolution. A user's workload description is converted into a schema-validated structured intent containing task characteristics, software requirements, resource constraints, and preferences. Candidate environments are then retrieved using two complementary retrieval channels: BM25-based sparse retrieval for exact technical terminology and dense semantic retrieval for semantically related workload descriptions. Their ranked results are combined using Reciprocal Rank Fusion (RRF). Retrieved candidates are subsequently evaluated against hard constraints, including GPU availability, minimum CPU and memory requirements, dataset size, and forbidden features. Candidates that violate hard constraints are excluded before deterministic preference ranking is applied.

The resulting recommendation is resolved through an administrator-owned candidate catalog rather than allowing the recommendation model to directly specify an image or Kubernetes

resource configuration. Approved notebook images are identified by immutable digest-pinned references, while resource profiles are restricted to administrator-defined configurations. A preview and confirmation mechanism binds the displayed recommendation to the authenticated user and to a single-use, time-limited token. The JupyterHub pre-spawn hook validates this state before applying the selected configuration, without performing new recommendation or model inference during pod creation. The system was evaluated through controlled experiment protocols covering recommendation behavior, system integration, resource-related measurements, and environment functionality. The evaluation distinguishes workload families as the semantic experimental unit and separates repeated executions used for stability analysis from independent recommendation cases.

The experimental evaluation examined recommendation quality and prompt robustness on development data, resource outcomes across 800 Kubernetes Pod trials, and image capability through deterministic probes and OCI layer accounting for four approved images. E1 estimates use 11 workload families with effective paired outcomes; E2 evaluates prompt robustness across 18 semantic families. The evaluation also established the functional integration of recommendation preview, policy validation, user confirmation, and pre-spawn application within JupyterHub, including enforcement of approved profiles and digest-pinned images. No human participant observations were collected.

These results indicate that intent-aware environment selection can be integrated into JupyterHub while maintaining a separation between language-based recommendation and authoritative infrastructure configuration. Hybrid retrieval provides a mechanism for combining exact technical vocabulary with semantic similarity, whereas deterministic constraints prevent soft relevance signals from overriding explicit resource or policy requirements. The architecture therefore provides a bounded form of decision support in which natural-language interaction reduces configuration effort without transferring deployment authority to the recommendation model. However, the available evidence does not establish statistical superiority over manual selection or alternative recommenders, nor does it demonstrate general performance across arbitrary multi-node, multi-tenant Kubernetes deployments. Human-user effectiveness and broader production validation remain subjects for further evaluation.

Keywords: JupyterHub, Kubernetes, Workload Intent, Resource Profile Recommendation, Hybrid Information Retrieval, Constraint-Based Selection, Notebook Environments.

AUTHORSHIP

I hereby declare that the work contained in this report is of my own and has not been previously submitted for a degree or diploma at this or any other higher education institution. To the best of my knowledge and belief, the report contains no materials previously published or written by another person except where due reference or acknowledgement is made.

Signature: _____

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my internship supervisor, Dr. Hoang Xuan Tung, for guidance, critical feedback, and encouragement throughout the development and evaluation of this work. I am also grateful to the lecturers and staff of the Faculty of Information Technology, for the technical and academic foundation that made this project possible.

I would also like to thank my friends, and family for their valuable discussions, encouragement, and practical support throughout this iterative process. I take full responsibility for any remaining errors, limitations, or interpretation choices in this report.

Contents

Abstract i

Authorship ii

Acknowledgements iii

List of Figures viii

List of Tables x

List of Algorithms xi

Abbreviations xii

Chapter 1. Introduction 1

1.1. Problem statement 1

1.2. Contributions and report overview 3

Chapter 2. Background 5

2.1. JupyterHub 5

2.2. Kubernetes execution model 8

2.2.1. Containers and pods 8

2.2.2. CPU and memory resources 9

2.2.3. Resource profiles 10

2.3. Notebook container images 11

2.4. Workload intent 12

2.5. Information retrieval 13

2.5.1. Sparse retrieval and BM25 15

2.5.2. Dense retrieval 15

2.5.3. Hybrid retrieval with reciprocal rank fusion 16

2.6. Constraint-based recommendation 17
Chapter 3. Proposed Method 19
3.1. Overview 19
3.2. Structured intent representation 21
3.3. Hybrid candidate retrieval 24
3.3.1. Retrieval pipeline architecture 24
3.3.2. Lexical query formulation 25
3.3.3. Semantic indexing and search 25
3.3.4. Rank fusion via RRF 25
3.4. Constraint filtering 26
3.5. Multi-criteria candidate ranking 29
3.5.1. Scoring formulation 30
3.5.2. Utility and resource cost modeling 30
3.6. JupyterHub integration and runtime enforcement 31
3.6.1. Policy validation and candidate resolution 32
3.6.2. Two-phase preview and confirmation 32
3.6.3. Pre-spawn binding and execution 34
3.6.4. Policy-bounded dynamic resource overlay 35
Chapter 4. Evaluation 37
4.1. Experimental design 37
4.1.1. Evaluation methodology and datasets 38
4.1.2. Implementation and reproducibility 39
4.2. Offline recommendation performance 40
4.2.1. End-to-end recommendation accuracy 40

4.2.2. Retrieval channel ablation study 44
4.2.3. Robustness to query variations 46
4.2.4. Accuracy-cost trade-off analysis 47
4.3. Cluster resource profiling 48
4.3.1. Pod scheduling and runtime overhead 48
4.3.2. Container resource usage profiling 51
4.4. Container image and storage analysis 53
4.4.1. Image capability probing 53
4.4.2. OCI layer and storage overhead 55
4.5. User study design 59
4.5.1. Study protocol and task design 59
4.5.2. Limitations and scope 60
Chapter 5. Conclusion 62
5.1. Conclusion 62
5.2. Future work 63
References 63

List of Figures

2.1. Subsystem architecture of JupyterHub showing the Hub, Configurable HTTP Proxy, Spawner, and single-user notebook servers [**?**] 7

2.2. Conceptual structure of a Kubernetes Pod encapsulating containers and shared storage volumes [**?**]..... 9

2.3. Layered container image architecture with stacked read-only layers and a top read-write container layer [**?**] 11

3.1. Overview of the proposed P2 intent-aware recommendation and execution architecture 20

3.2. Hybrid candidate retrieval pipeline combining sparse lexical and dense semantic channels with Reciprocal Rank Fusion 24

3.3. Deterministic hard constraint evaluation decision tree and pruning logic 28

3.4. Two-phase preview, user confirmation, and pre-spawn binding lifecycle 33

4.1. Multi-stream experimental evaluation framework of Intent-Spawner, separating offline recommendation benchmarks, Kubernetes cluster trials, container capability sandboxes, and OCI manifest storage accounting..... 38

4.2. Offline recommendation quality and robustness metrics in Stream E1, comparing P1 (rule-based) and P2 (Intent-Spawner) with macro-averaged point estimates and 95% percentile bootstrap confidence intervals..... 43

4.3. Candidate retrieval channel ablation across Recall@1, Recall@3, and Recall@5, comparing Sparse BM25, Dense Semantic (all-MiniLM-L6-v2), and Hybrid RRF fusion. 45

4.4. Observed Kubernetes Pod execution reliability across 800 real trials in Stream E4 (left), and the conceptual trade-off between OOM risk and resource savings across profile allocation strategies (right). 50

4.5. Locally observed memory envelopes (P2 Safe Envelope) compared against Static Small (256 MiB) and Static Large (2048 MiB) profiles across 20 workload families in the E4 experiment. 52

4.6. Three-tier container image verification architecture in Stream E5, separating gold-label correctness (Dimension A), catalog metadata soundness (Dimension B), and in-container runtime capability execution (Dimension C). 54

4.7. Cumulative compressed OCI layer storage across the four-image catalog prefix in Stream E5, demonstrating a 27.61% deduplication saving achieved through shared base layers. 57

4.8. Marginal unique compressed layer storage introduced by each container image in Stream E5, showing shared base layer reuse between general-purpose and deep learning images. 58

4.9. Pairwise shared layer storage heatmap (GiB) among the four approved container images, highlighting substantial layer inheritance between data science and deep learning environments. 59

List of Tables

3.1. Specification of the StructuredIntent data contract and normalization rules……22

3.2. Deterministic hard constraint evaluation rules and pruning conditions……27

4.1. Experiment streams, evaluated constructs, and empirical evidence scope ……39

4.2. Observed E1 offline recommendation quality estimates for P1 and P2 (Local vs. LLM Mode) ……42

4.3. Observed E2 semantic and linguistic robustness metrics on the development split47

4.4. Observed E4 Kubernetes Pod execution outcomes across 800 trials (OrbStack environment)……50

4.5. E5 container image functional capability probe evaluation results……55

4.6. Observed compressed OCI layer-byte accounting and deduplication across four images ……56

4.7. Marginal unique compressed layer bytes introduced by image order ……57

List of Algorithms

3.1. Structured intent extraction, normalization, and validation procedure 23

3.2. Reciprocal Rank Fusion for hybrid candidate retrieval 26

3.3. Deterministic candidate evaluation, multi-criteria scoring, and tie-breaking 31

3.4. JupyterHub pre-spawn policy validation and deterministic resource binding 34

Abbreviations

API	Application Programming Interface
BM25	Best Matching 25
CPU	Central Processing Unit
GPU	Graphics Processing Unit
HCI	Human-Computer Interaction
IR	Information Retrieval
LLM	Large Language Model
MRR	Mean Reciprocal Rank
nDCG	Normalized Discounted Cumulative Gain
OCI	Open Container Initiative
POD	Plain Old Data / Kubernetes Pod (context dependent)
RRF	Reciprocal Rank Fusion
RQ	Research Question
TTL	Time to Live
VPA	Vertical Pod Autoscaler

Chapter 1.

Introduction

1.1. Problem statement

The development of interactive computing and container orchestration has progressively changed how computational environments are provided to users. In 2014, the Jupyter Project branched off from the IPython project as a project aimed at supporting interactive data science and scientific computing across all programming languages. Around the same time that Jupyter was emerging, the open source project Kubernetes was introduced as a system to manage containerized workloads. Kubernetes received its first public commit in June 2014 and reached 1.0 in July 2015. JupyterHub, a multi-user version of the Jupyter notebook, was released as a preview in March 2015. The combination of JupyterHub and container orchestration technologies such as Kubernetes has since provided a practical foundation for delivering isolated and centrally managed notebook environments to groups of users. Today, JupyterHub explicitly supports Kubernetes-based deployments and is designed to provide users with computational environments and resources while allowing administrators to manage the underlying infrastructure.

Interactive computational environments have become an important part of modern data analysis, scientific computing, education, and machine learning. Jupyter notebooks are particularly useful in these settings because they allow users to combine executable code, computational results, and explanatory text within a single interactive environment. In multi-user deployments, however, providing notebook environments is not only a matter of running the notebook application itself. The platform must also manage users, compute resources, storage, software environments, and isolation between workloads. JupyterHub provides a common platform for deploying and managing multi-user Jupyter environments, while Kubernetes can provide the underlying infrastructure for scalable and isolated

execution. In a typical JupyterHub deployment on Kubernetes, KubeSpawner creates a Kubernetes Pod for each user's notebook server and can expose a set of administrator-defined resource profiles and container images. Such profiles allow administrators to control the available CPU, memory, accelerators, and software environments while preventing users from directly requesting arbitrary infrastructure configurations. This administrator-controlled approach provides an important operational boundary, but it also introduces a usability problem. Users generally describe their requirements in terms of the work they want to perform rather than in terms of infrastructure configuration. For example, a user may know that they need to train a machine-learning model on a large dataset, process images, or perform data analysis with a particular framework. The platform, however, may require the user to select an appropriate resource profile and notebook image from a predefined catalog. Consequently, users may need to understand technical differences between available environments before they can start their actual work. This creates a gap between workload intent and infrastructure configuration. A user understands the intended computation, while the platform requires a concrete profile and image selection. Bridging this gap is particularly relevant when a catalog contains multiple environments with different resource capacities and software capabilities.

One way to present users with the options of administrator-defined solution profiles is to create a simple list that users can select from, one at a time. But that way users will have to interpret the solution catalog for themselves. A simple name for a profile does not necessarily imply that the solution will be suitable for a given workload. And relying on familiar software names to pick environments does not necessarily guarantee that the chosen environment meets the user's explicit resource or capability requirements. Automating this decision is not straightforward either. A purely rule-based approach can provide deterministic and easily explainable decisions, but its ability to handle the variety of natural-language workload descriptions may be limited. Conversely, semantic retrieval or language-model-based approaches can improve the ability to interpret natural-language requests, but semantic relevance alone cannot guarantee that a candidate satisfies mandatory requirements or conforms to administrator-defined policies. The problem addressed by this internship project is therefore the design of a context-aware, policy-

bounded mechanism for recommending an appropriate JupyterHub resource profile and notebook image from an administrator-controlled catalog based on a user's workload intent. The problem is deliberately narrower than general cloud resource optimization. This system does not attempt to predict exactly how much resources a given workload will use to run, replace the scheduling provided by Kubernetes, or try to optimize the whole cluster. What this system tries to do is decide on which of the pre-authorized environments by an administrator a user can select from their descriptions of intended workloads, and lists all relevant environments that satisfy all given requirements and constraints of the platform.

1.2. Contributions and report overview

The main contributions of this internship project are twofold. First, the project develops a context-aware resource profile selection approach that connects natural-language workload descriptions with a finite catalog of administrator-approved JupyterHub environments. The approach considers workload intent, explicit resource requirements, and the available environment catalog as relevant context during the selection process. This allows users to describe their computational needs at the workload level while keeping the actual resource environments within the set of configurations defined and controlled by the platform administrator. Second, the report presents a preliminary evaluation framework for assessing the resource profile selection approach. Rather than relying on a single recommendation metric, the evaluation considers several aspects of system behavior, including recommendation quality, robustness to variations in workload descriptions, resource-related behavior, notebook-image functionality, and practical integration with the JupyterHub environment. This evaluation is intended to provide a view of the effectiveness and practical applicability of the proposed approach.

The remainder of the report is organized into four chapters. Chapter 2 presents the technical and conceptual foundations required for the proposed work. It introduces JupyterHub and Kubernetes-based notebook environments, resource profiles and container

images, workload intent, information retrieval, semantic retrieval, and recommendation-related concepts. These foundations establish the terminology and technical background used throughout the subsequent chapters. Chapter 3 presents the design of the proposed resource profile selection approach in detail. It describes the overall system architecture, workload-intent representation, candidate retrieval and ranking process, constraint handling, administrator-controlled environment catalog, and integration with the JupyterHub pre-spawn workflow. The chapter also discusses the main design decisions and defines the scope and boundaries of the proposed system. Chapter 4 describes the implementation and experimental methodology used to assess the proposed approach. It presents the experimental setup, evaluation scenarios, baselines, and evaluation metrics, followed by the experimental results and their discussion. The chapter also examines the limitations and scope of the evidence obtained from the experiments. Chapter 5 summarizes the work and its main findings, discusses the limitations of the current approach, and presents directions for future development and further evaluation. Overall, this internship project investigates the use of workload intent as an interface for selecting appropriate notebook environments in JupyterHub on Kubernetes. The central aim is to reduce the gap between user-level descriptions of computational tasks and infrastructure-level environment selection while maintaining administrator control and adherence to explicit platform constraints.

Chapter 2.

Background

This chapter introduces the technical concepts and methods required to understand the design of the proposed system. The discussion starts with interactive computational environments and their multi-user management through JupyterHub. It then describes Kubernetes-based execution, resource profiles, and notebook container images. The chapter subsequently introduces workload intent representation and the information retrieval techniques used for matching a natural-language request against a catalog of computational environments. Finally, constraint-based recommendation is discussed as the mechanism for ensuring that retrieved candidates satisfy explicit resource and environment requirements. The purpose of this chapter is to establish the concepts used later in the system design and evaluation. The implementation-specific architecture of the proposed system is presented separately in Chapter 3.

2.1. JupyterHub

Interactive computational environments are widely used for data analysis, scientific computing, machine learning, education, and software development. Instead of separating source code, execution, and results into independent tools, an interactive notebook allows these elements to be combined in a single computational document [**?**]. Jupyter Notebook is one of the most widely adopted environments following this model. At a basic level, a notebook server provides an interface through which a user can execute code interactively and inspect the resulting output. The Jupyter architecture separates the user-facing notebook interface from the kernel responsible for executing computations. Consequently, the computational environment can be configured independently of the browser-based interface. This separation is particularly useful in environments where users have different computational requirements. A lightweight educational notebook

may require only a basic Python environment, whereas a machine-learning workload may require additional libraries, larger amounts of memory, or specialized hardware accelerators. Therefore, providing an interactive notebook service at scale requires more than simply making the Jupyter interface available: the underlying execution environment must also be provisioned and isolated appropriately.

JupyterHub extends the Jupyter ecosystem from a single-user environment to a multi-user service [**?**]. It provides authentication, user management, and the creation of individual single-user notebook servers. The JupyterHub documentation describes the spawner as the component responsible for starting the single-user server and allocating its execution environment [**?**]. Conceptually, a JupyterHub deployment consists of several cooperating components, as illustrated in Figure 2.1. The Hub coordinates authentication through an Authenticator and manages user sessions recorded in a state database, while a Configurable HTTP Proxy routes incoming web requests either to the Hub for login operations or directly to the running notebook server of an authenticated user. A Spawner is responsible for creating and managing each user's computational environment upon demand. The resulting architecture separates service management from the actual execution of notebook workloads.

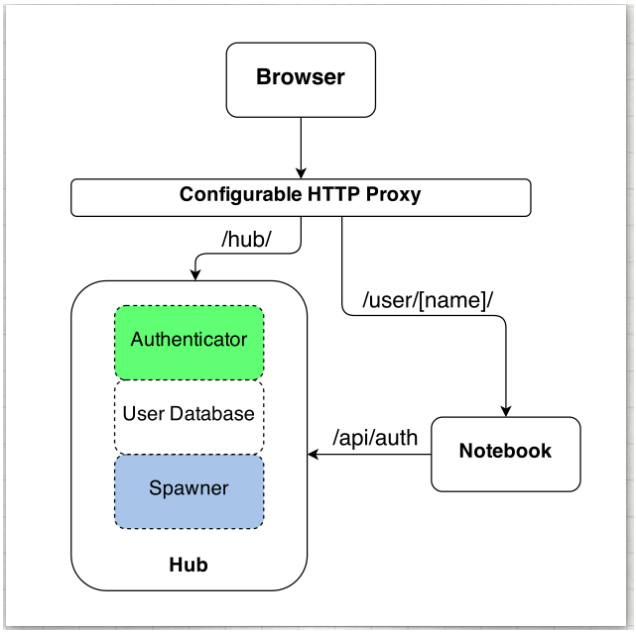


Figure 2.1. Subsystem architecture of JupyterHub showing the Hub, Configurable HTTP Proxy, Spawner, and single-user notebook servers [**?**]

This distinction is important for resource-aware notebook provisioning. The Hub does not itself perform user computations. Instead, the spawner translates the desired user environment into the configuration required by the underlying execution platform, creating and tearing down instances as users arrive and leave.

JupyterHub also provides an abstract Spawner interface for starting, polling, and stopping single-user notebook servers [**?**]. Different spawner implementations can use different execution backends, including local operating system processes, container runtimes, and Kubernetes clusters. In a Kubernetes deployment, KubeSpawner creates each user's notebook server as a dedicated Kubernetes workload [**?**, **?**]. A spawner can also expose configuration options through the JupyterHub spawn form. Such options may represent CPU and memory allocations, container images, or a predefined combination of settings called a profile. JupyterHub's documentation explicitly describes profiles as a mechanism through which users can select a bundle of configuration settings for their notebook server [**?**]. A profile therefore provides an abstraction between low-level infrastructure configuration and user choice. Rather than requiring users to understand the exact Kubernetes resource specification, an administrator can define a small set of named

environments such as *Small*, *Medium*, and *Large*. The user then selects the environment that appears appropriate for the workload. However, manual selection often imposes substantial cognitive burden and decision difficulty on users who lack infrastructure expertise [**?**], underscoring the need for assistive decision-support mechanisms in complex choice environments [**?**]. This mechanism provides an important foundation for automated environment recommendation. The recommendation problem can be formulated as selecting one of a finite set of administrator-defined environments rather than generating arbitrary infrastructure configurations dynamically.

2.2. Kubernetes execution model

2.2.1. Containers and pods

Kubernetes provides an orchestration layer for containerized applications. The basic execution unit relevant to a JupyterHub deployment is the Pod [**?**]. A Pod contains one or more containers that share storage volumes, a network namespace with a single IP address, and specifications on how to run the workloads. Figure 2.2 illustrates the conceptual structure of a Pod encapsulating containerized components and shared storage. For notebook platforms, a user's single-user server is represented by a dedicated Pod where the container image determines the software environment available to the user, while the Kubernetes resource configuration determines the computational envelope allocated to the workload.

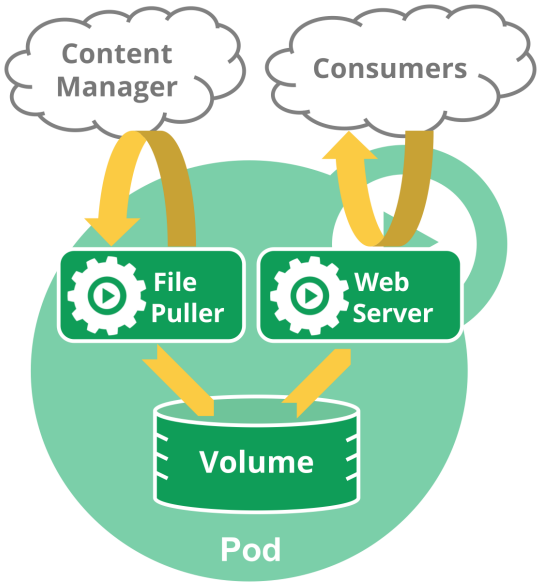


Figure 2.2. Conceptual structure of a Kubernetes Pod encapsulating containers and shared storage volumes [**?**]

This separation establishes two complementary dimensions of environment selection. On the software dimension, the container image defines the runtime environment, pre-installed frameworks, domain-specific libraries, and underlying system dependencies. On the hardware resource dimension, the allocated CPU, memory, specialized hardware accelerators or GPUs [**?**], and associated platform limits establish the computational envelope in which the workload executes. Consequently, an automated environment recommendation system must jointly consider both dimensions rather than treating resource sizing and software selection as unrelated choices.

2.2.2. CPU and memory resources

Kubernetes allows resource requests and limits to be specified for containers [**?**]. A resource request represents the amount of a resource guaranteed and considered by the Kubernetes scheduler during node placement, while a resource limit establishes an upper bound on resource consumption enforced at runtime [**?**]. For CPU and memory, these settings carry distinct operational implications. The Kubernetes scheduler uses resource requests when determining whether a Pod can be placed on a node without violating capacity constraints. In contrast, resource limits are enforced by kernel control groups:

CPU limits can lead to CPU throttling when exceeded, whereas memory limits cause out-of-memory (OOM) terminations when memory pressure forces a container beyond its configured boundary [**?**, **?**]. Furthermore, aggregate namespace allocations can be restricted using ResourceQuota objects [**?**]. Consequently, resource configuration must balance two competing operational concerns. If a profile provides too few resources, a workload may fail through OOM kills or suffer severe performance degradation. If it provides substantially more resources than necessary, the environment consumes cluster capacity that could otherwise serve other concurrent users, leading to widespread over-allocation and resource stranding observed in production multi-tenant clusters [**?**, **?**, **?**]. For a multi-user notebook platform, resource-profile selection is thus a practical systems decision problem rather than merely a cosmetic interface preference.

2.2.3. Resource profiles

A resource profile is a predefined mapping from a human-readable environment choice to concrete infrastructure configurations. In the context of the proposed system, the repository defines three representative static profiles. The *Small* profile is intended for basic Python programming and lightweight notebook workloads, offering modest resource allocations to maximize cluster density. The *Medium* profile is configured for moderate data exploration, tabular data processing, and visualization tasks that require additional memory headroom. The *Large* profile is designed for compute- and memory-intensive workloads, such as local model training or iterative numerical experimentation. The profiles differ systematically in their CPU and memory allocations. For example, the repository configuration defines the Small profile with a CPU guarantee of 0.1 cores, a CPU limit of 0.5 cores, a memory guarantee of 256 MiB, and a memory limit of 384 MiB. The Medium and Large profiles provide progressively larger resource envelopes. These values represent administrative policy choices rather than universal definitions of workload scale. Their primary role is to provide a finite, administrator- controlled candidate space from which a recommendation can be selected. This distinction is central to the system developed in this internship project. The recommendation component does not invent arbitrary CPU or memory values. Instead, it selects an environment from the administrator-

defined catalog and verifies the selected candidate against explicit policy and capacity constraints.

2.3. Notebook container images

A container image packages the software environment required to execute a workload, encompassing the base operating system runtime, language interpreters, scientific libraries, machine-learning frameworks, and supporting system packages [**?**]. As shown in Figure 2.3, container images follow a layered filesystem architecture where immutable, content-addressed read-only layers are stacked together, and a thin read-write layer is added at runtime when a container is instantiated [**?**]. This layered design enables efficient image distribution and local layer sharing across diverse notebook configurations, as empirical analyses of container workloads and image registries demonstrate significant redundancy and reuse potential across layer stacks [**?**, **?**].

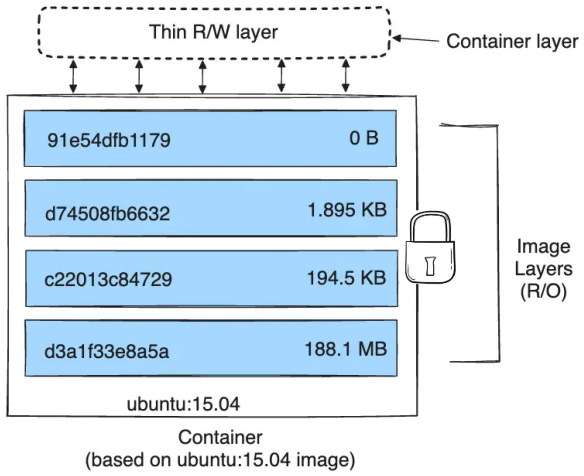


Figure 2.3. Layered container image architecture with stacked read-only layers and a top read-write container layer [**?**]

Different computational workloads inherently require different software stacks. A general programming task may require only a minimal Python runtime, while a data-analysis workflow requires packages such as NumPy, pandas, SciPy, and scikit-

learn. Deep-learning workloads instead depend on specialized frameworks such as PyTorch or TensorFlow alongside accelerator drivers and CUDA runtime libraries. The proposed system models an execution environment as a coupled pair consisting of an approved resource profile and an administrator- reviewed notebook image, allowing the recommendation process to account simultaneously for computational resources and software dependencies.

Allowing an automated system to freely generate or pull arbitrary container image references introduces significant operational, security, and stability risks. An unvetted image may contain malicious code, outdated software, or incompatible dependencies that disrupt cluster operation. For this reason, the platform relies on an administrator-owned image catalog. The representative catalog entries include `minimal-python` for general Python programming and lightweight workloads, `scipy-data-science` for scientific computing, visualization, and conventional machine learning, `pytorch-deep-learning` for PyTorch-based deep-learning tasks, and `tensorflow-deep-learning` for TensorFlow- and Keras-based workflows. Each catalog entry is annotated with structured metadata detailing supported task types, frameworks, libraries, hardware capabilities, and suitability tags. This metadata provides a searchable semantic representation of the available environments, while also establishing an authoritative trust boundary: the recommendation algorithm is restricted to selecting among administrator-reviewed candidates rather than producing unverified container references.

2.4. Workload intent

Traditional resource-profile selection exposes infrastructure concepts directly to the end user. A user must determine whether a workload requires a small, medium, or large environment and separately identify which container image contains the appropriate software packages. This approach assumes that users can accurately translate their computational requirements into infrastructure specifications. In practice, users describe their goals in terms of the task they wish to perform. For example, a user may describe

a workload as “I want to train a PyTorch model on a large dataset.” Such a description conveys information regarding the computational task, framework, and relative data scale, but it does not specify concrete Kubernetes CPU or memory allocations. The central abstraction used by the proposed system is therefore *workload intent*, drawing on the principles of intent-based computing where high-level user goals are translated into system configurations [**?**, **?**, **?**]. Workload intent provides a structured representation of what the user wants to accomplish and what operational properties the resulting computational environment must satisfy.

Natural-language expressions provided by human users are inherently heterogeneous, unstructured, and ambiguous. A user describing a computational goal typically focuses on domain objectives (such as the machine learning framework, analytical procedure, or estimated dataset size) rather than low-level infrastructure attributes. Translating such unstructured natural-language requests into executable system configurations or platform commands is a recognized challenge in systems interfaces [**?**]. Intent-based computing formalizes this paradigm by decoupling user intent specification from underlying execution configuration [**?**, **?**]. Under this framework, high-level intent is translated into a validated, declarative contract that captures functional goals, explicit constraints, and user preferences. By establishing a structured intermediate representation, intent-based systems isolate the stochastic nature of natural language interpretation from deterministic execution engines. This structured bridge enables automated systems to parse user goals, validate semantic consistency, and apply rigorous policy enforcement without giving natural-language interfaces direct access to infrastructure provisioning mechanisms [**?**].

2.5. Information retrieval

The environment catalog used by the proposed system is a finite collection of administrator-defined computational environments. Each environment is associated with structured metadata describing properties such as supported workload types, frameworks, libraries, capabilities, and resource characteristics. The recommendation problem can

therefore be viewed as a search problem over this catalog: given a user's workload description, the system must identify the environments whose metadata is most relevant to the request. This perspective motivates the use of *information retrieval* (IR). Rather than requiring every possible natural-language expression to be explicitly encoded as a rule, retrieval allows the system to search the existing environment catalog and produce a ranked set of potentially relevant candidates.

Let

$$\mathcal{D} = \{d_1, d_2, \ldots, d_n\}$$

represent the administrator-defined environment catalog and let q represent the user's workload intent. A retrieval function produces a ranked list

$$R(q) = (d_{(1)}, d_{(2)}, \ldots, d_{(k)}),$$

where candidates appearing earlier in the list are considered more relevant to the query.

The use of retrieval is particularly appropriate because the system does not need to construct a new environment for every request. Instead, it searches within a controlled catalog and identifies candidates that already exist. This also preserves the administrator-defined boundary of the system: retrieval determines which existing environments are relevant, while subsequent constraint evaluation determines which of them are actually feasible. A straightforward approach would be to match the words in the user's request directly against the metadata of available environments. Although simple and interpretable, exact keyword matching is sensitive to differences in vocabulary. For example, a user may describe a workload as "training a neural network", while an environment may be described using terms such as "deep learning", "PyTorch", or "model training". These expressions may refer to related workloads without sharing exactly the same words. The opposite problem can also occur. Technical terms such as "PyTorch", "TensorFlow", or "CUDA" carry specific meanings and should not necessarily be treated as interchangeable with loosely related concepts. Consequently, an effective retrieval mechanism should provide both lexical precision for explicit technical terminology and tolerance to variations

in natural-language expression. This motivates the use of two complementary retrieval approaches: sparse lexical retrieval and dense semantic retrieval.

2.5.1. Sparse retrieval and BM25

Sparse information retrieval methods represent queries and candidate documents primarily through their lexical features. They are particularly effective when important concepts are explicitly expressed using the same terms in the query and candidate metadata.

BM25 is a widely used probabilistic ranking function [**?**]. Given a query Q and document D, its score can be expressed as

$$\mathrm{BM25}(D,Q) = \sum_{t \in Q} \mathrm{IDF}(t) \cdot \frac{f(t,D)(k_1+1)}{f(t,D) + k_1 \left(1 - b + b\frac{|D|}{\mathrm{avgdl}}\right)},$$

where $f(t,D)$ is the frequency of term t in document D, $|D|$ is the document length, avgdl is the average document length across the corpus, $\mathrm{IDF}(t)$ is the inverse document frequency of term t, and k_1 and b control term-frequency saturation and length normalization, respectively. BM25 is useful for the proposed environment catalog because explicit technical vocabulary can be highly informative. For example, a request containing "pandas" can be matched directly against an environment whose metadata contains "pandas". Similarly, an explicit mention of "PyTorch" can provide strong evidence for a PyTorch-oriented environment. However, BM25 primarily relies on lexical overlap. If a user and an environment describe the same concept using substantially different vocabulary, their lexical similarity may be low. Therefore, BM25 alone does not fully address the variability of natural-language workload descriptions.

2.5.2. Dense retrieval

Dense retrieval addresses vocabulary mismatch by representing queries and candidate environments as vectors in a continuous embedding space [**?**, **?**, **?**]. Let $\mathbf{q} = f(q)$ be the embedding of a query and $\mathbf{d} = f(d)$ be the embedding of a candidate environment.

A common similarity measure is cosine similarity:

$$\mathrm{sim}(\mathbf{q},\mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\|\|\mathbf{d}\|}.$$

Candidates with larger similarity values are considered semantically closer to the query.

The main advantage of dense retrieval is its ability to capture semantic relationships beyond exact word overlap. For example, a query describing “neural-network training” may be considered semantically related to an environment described as supporting “deep learning” and “model training”, even if the two descriptions do not share all of their terms. Dense retrieval alone, however, also has limitations for this application. Embedding similarity may treat technically related expressions as similar even when an exact distinction is important. For example, different machine learning frameworks may be semantically related but are not necessarily interchangeable when a user explicitly requires one particular framework. The ranking produced by dense retrieval can also depend strongly on the embedding model and the representation of the candidate metadata. Therefore, dense retrieval is useful for discovering semantic relationships, but it should not be treated as the sole mechanism for interpreting explicit technical requirements.

2.5.3. Hybrid retrieval with reciprocal rank fusion

The characteristics of the environment recommendation problem suggest that neither sparse nor dense retrieval is sufficient on its own. Sparse retrieval provides strong lexical sensitivity. This is valuable when a user explicitly mentions technical entities such as a programming framework, library, or capability. Dense retrieval provides semantic flexibility and can help identify relevant environments when the user’s wording differs from the wording used in the catalog. These properties are complementary. Sparse retrieval is effective for explicit terminology and exact technical concepts, dense retrieval is effective for semantic similarity and variation in natural-language descriptions, and hybrid retrieval combines these signals to obtain a broader candidate set while retaining sensitivity to explicit technical vocabulary. Hybrid retrieval is therefore selected as the retrieval strategy for the proposed system. The goal is not to allow semantic similarity to override

explicit requirements, but to improve the identification of relevant candidates before deterministic constraint evaluation. This distinction is important. Retrieval is responsible for answering "*Which environments appear relevant to this workload?*", whereas the subsequent constraint layer answers "*Which of these environments are actually allowed and feasible?*". Separating these two questions prevents a high semantic similarity score from being treated as sufficient evidence that an environment satisfies a mandatory requirement.

Let $r_s(d)$ and $r_d(d)$ denote the ranks assigned to candidate d by the sparse and dense retrieval methods, respectively. Their rankings can be combined using Reciprocal Rank Fusion (RRF) [**?**]:

$$\mathrm{RRF}(d) = \sum_{m \in \{s,d\}} \frac{w_m}{k_{\mathrm{RRF}} + r_m(d)},$$

where w_m is the weight assigned to retrieval method m, and k_{RRF} is a constant controlling the contribution of lower-ranked candidates.

RRF is suitable for combining the two retrieval channels because it operates on rankings rather than requiring their raw scores to be directly comparable. This is useful in this setting because BM25 scores and embedding similarities have different numerical scales and interpretations. The resulting hybrid ranking provides the candidate set for the next stage of the recommendation pipeline. Importantly, this ranking is not itself the final provisioning decision. Retrieved candidates are subsequently evaluated against hard resource and software constraints and may then be ordered using soft preferences.

2.6. Constraint-based recommendation

Information retrieval assesses lexical and semantic relevance, but relevance alone is insufficient for decision-support and resource provisioning systems. A candidate solution may demonstrate high textual affinity to a user query while failing to meet mandatory operational requirements, physical prerequisites, or administrative policies. For domains governed by explicit rules, recommendation systems must incorporate constraint

satisfaction mechanisms to guarantee solution validity. Constraint-based recommendation represents a major paradigm within knowledge-based recommender systems, designed for complex problem domains where items must satisfy strict technical specifications, compatibility rules, and domain constraints [**?**]. Unlike collaborative filtering or pure content-based retrieval models that rely primarily on statistical correlation or historical usage patterns, constraint-based approaches model the recommendation task by combining constraint satisfaction with multi-attribute utility optimization.

In constraint-based recommendation theory, domain criteria are partitioned into two fundamental categories. Hard constraints are strict, non-negotiable operational requirements that define the boundary of valid solutions. A candidate item must satisfy all hard constraints simultaneously to be considered feasible. In technical systems, hard constraints prevent severe runtime failures (such as resource exhaustion or missing runtime dependencies), because violations cannot be compensated for or traded off against high relevance scores. On the other hand, soft constraints and preferences are desirable but optional criteria that express user affinities, quality indicators, or secondary optimization goals. Soft preferences are evaluated across the set of verified feasible solutions using multi-attribute utility functions, guiding the ranking of viable candidates without compromising feasibility [**?**, **?**]. Furthermore, evaluating such decision-support systems requires examining not only algorithmic accuracy but also user perception, trust, and decision quality, as established in user-centric recommender system evaluation frameworks [**?**, **?**]. Structuring the decision-support pipeline into a two-phase architecture candidate retrieval followed by deterministic constraint evaluation and utility ranking ensures both search efficiency and operational correctness. Candidate retrieval efficiently narrows a large catalog down to a promising subset, while subsequent constraint filtering guarantees that only viable, policy-compliant candidates proceed to final ranking and deployment.

Chapter 3.

Proposed Method

3.1. Overview

This chapter describes the design and formal model of our intent- and context-aware environment selector for JupyterHub on Kubernetes. In multi-tenant clusters, data scientists often struggle to choose the right container image and resource allocation. They must navigate dozens of software stacks, GPU configurations, and CPU/memory limits without knowing exactly what their code needs. Our system, named P2, automates this selection from free-form natural language.

We base P2 on a straightforward design principle: language models interpret user intent, but deterministic rules control infrastructure. Letting a generative model write Kubernetes Pod manifests directly is dangerous. Models hallucinate package names, produce invalid resource requests, and cannot guarantee policy compliance. Instead, P2 confines language understanding to semantic extraction. Downstream retrieval, constraint pruning, scoring, and pod configuration operate strictly over an administrator-approved candidate catalog.

Formally, P2 maps an incoming request to an authoritative deployment through a chain of deterministic stages:

$$x \longrightarrow I \longrightarrow (L_s, L_d) \longrightarrow L_f \longrightarrow F \longrightarrow c^* \longrightarrow \mathbf{e}^*,$$

where x is the raw natural-language query; I is the validated, schema-compliant intent contract; L_s and L_d denote candidate shortlists from sparse lexical and dense semantic search; L_f is the merged ranking from Reciprocal Rank Fusion; F is the feasible subset

surviving hard constraint checks; c^* represents the top candidate chosen by multi-criteria ranking; and $\mathbf{e}^*$ is the concrete Kubernetes configuration applied at spawn time.

Figure 3.1 shows the end-to-end architecture of P2 and the trust boundary between recommendation logic and runtime execution.

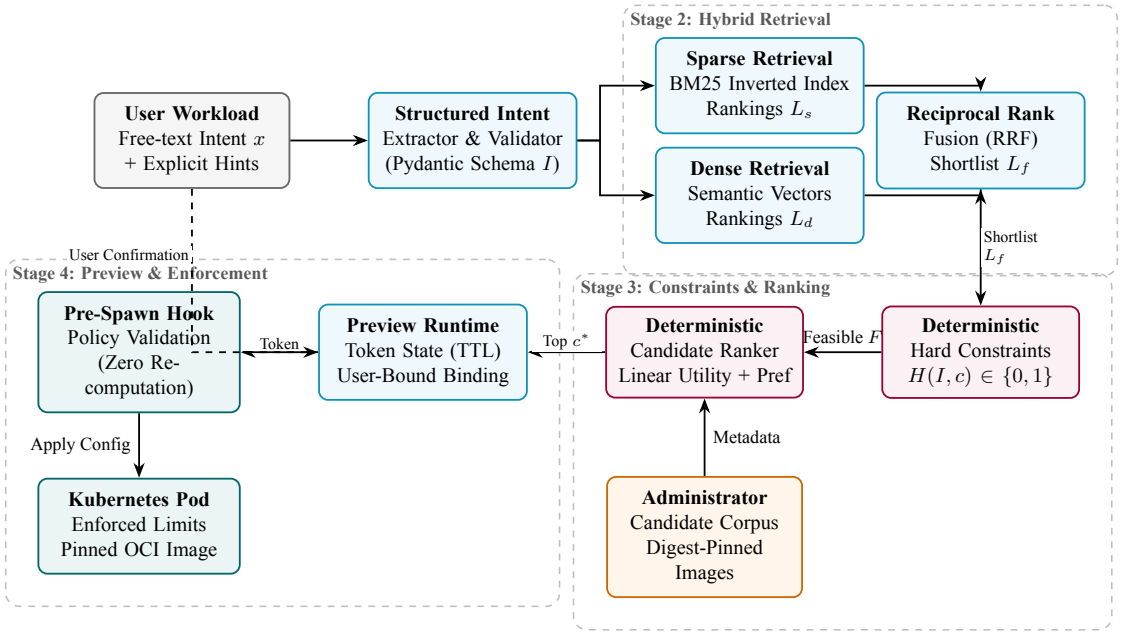


Figure 3.1. Overview of the proposed P2 intent-aware recommendation and execution architecture

The pipeline consists of five core components arranged in a feed-forward workflow:

1. **Intent Interface and Extractor:** Parses the user prompt x and any manual UI hints into a typed `StructuredIntent` object (I).
2. **Candidate Catalog and Hybrid Retriever:** Indexes verified image-profile pairings. It queries them across parallel sparse (BM25) and dense (vector embedding) channels, producing shortlists L_s and L_d, which fuse into L_f.
3. **Constraint Evaluator and Deterministic Ranker:** Prunes any candidate that violates mandatory GPU, CPU, memory, or software requirements to form the feasible set F. It then scores the survivors using retrieval rank, user preferences, and resource cost to select c^*.

4. **Policy Validator and Preview Runtime:** Checks c^* against cluster allowlists. If valid, the system creates a time-limited preview record identified by a single-use token.
5. **JupyterHub Pre-Spawn Hook:** Intercepts the final notebook launch. It verifies the preview token and binds the confirmed container image and resource limits into KubeSpawner.

This workflow keeps recommendations advisory. Users retain full control to inspect, confirm, or override the chosen environment before any pod starts.

3.2. Structured intent representation

Free-form workload requests are noisy. A user prompt often blends high-level project goals (“fine-tune a vision model”), concrete library choices (“use PyTorch and timm”), and specific hardware constraints (“needs at least 16GB RAM and a GPU”). Passing such unstructured queries directly into candidate search or cluster controllers couples fuzzy language understanding with strict infrastructure requirements.

To prevent this coupling, P2 introduces an intermediate schema called `StructuredIntent`. This contract isolates the extraction stage from ranking and execution. It organizes workload properties into clear functional dimensions: task categories, required/preferred/forbidden features, framework and library dependencies, numeric resource constraints, ambiguity flags, normalized query text, and extraction provenance metadata.

We strictly separate *required* from *preferred* attributes. A required attribute is a hard precondition. If an environment lacks a required library or GPU capability, the evaluator prunes it immediately. In contrast, a preferred attribute reflects soft user affinity. It does not eliminate candidates; instead, it provides a scoring boost to otherwise feasible profiles. Table 3.1 summarizes the schema specification and validation rules.

Table 3.1. Specification of the StructuredIntent data contract and normalization rules

Field Name	Data Type	Normalization Rule	Evaluation Role
`task_types`	Tuple[`TaskType`]	Enum validation, deduplicated	Sparse/dense query context
`required_features`	Tuple[str]	NFKC, case-folded, sorted	Hard constraint filter (H)
`preferred_features`	Tuple[str]	NFKC, case-folded, sorted	Soft preference score (S_{pref})
`forbidden_features`	Tuple[str]	NFKC, case-folded, sorted	Hard exclusion predicate (H)
`required_frameworks`	Tuple[str]	NFKC, case-folded, sorted	Hard constraint filter (H)
`preferred_frameworks`	Tuple[str]	NFKC, case-folded, sorted	Soft preference score (S_{pref})
`required_libraries`	Tuple[str]	NFKC, case-folded, sorted	Hard constraint filter (H)
`preferred_libraries`	Tuple[str]	NFKC, case-folded, sorted	Soft preference score (S_{pref})
`gpu_requirement`	`GPURequirement`	Enum (`required/preferred/not`)	Hard constraint / preference
`minimum_cpu_cores`	Optional[float]	Finite non-negative float	Hard lower-bound constraint
`minimum_memory_gb`	Optional[float]	Finite non-negative float	Hard lower-bound constraint
`dataset_size_gb`	Optional[float]	UI precedence, finite float	Hard memory headroom bound
`ambiguities`	Tuple[str]	Printable characters, sorted	Diagnostic explanation
`normalized_query`	str	Whitespace collapsed	Sparse BM25 retrieval token
`extraction_provenance`	`Provenance`	Extractor ID, mode, hash	Security and audit tracking

Algorithm 3.1 outlines how P2 extracts, normalizes, and validates user requests into this structured contract.

Algorithm 3.1. Structured intent extraction, normalization, and validation procedure

Require: Raw user request string x, explicit UI parameters $H_{\text{user}} = \langle \text{min_cpu}, \text{min_ram}, \text{dataset_size}, \text{gpu} \rangle$

Ensure: Validated, immutable structured intent contract I

$I_{\text{raw}} \leftarrow \text{ExtractSemantics}(x)$ ▷ *Structured LLM parser or regex fallback*

$I_{\text{norm}} \leftarrow \text{CanonicalizeUnicodeAndText}(I_{\text{raw}})$

$I_{\text{norm}}.\text{req_feats} \leftarrow \text{DeduplicateAndSort}(I_{\text{norm}}.\text{req_feats})$

$I_{\text{norm}}.\text{pref_feats} \leftarrow \text{DeduplicateAndSort}(I_{\text{norm}}.\text{pref_feats}) \setminus I_{\text{norm}}.\text{req_feats}$ ▷ *Required takes precedence*

if $I_{\text{norm}}.\text{req_feats} \cap I_{\text{norm}}.\text{forbid_feats} \neq \emptyset$ **then**

    return error(`InvalidContract: Contradictory required and forbidden features`)

end if

$R \leftarrow \text{OverrideResourceConstraints}(I_{\text{norm}}.\text{resource_constraints}, H_{\text{user}})$ ▷ *UI hints override extracted values*

$I \leftarrow \text{FreezeContract}(I_{\text{norm}}, R, \text{schema_version} = \text{"1.0.0"})$

return I

Before retrieval runs, P2 canonicalizes all string and collection fields. It applies Unicode NFKC normalization, strips punctuation, folds case, and collapses whitespace. It also deduplicates and sorts lists of frameworks and libraries. This step ensures that cosmetic variations do not affect retrieval results. For example, `"Scikit-Learn"`, `"scikit_learn"`, and `" scikit-learn "` map to the same token.

The normalizer also resolves conflicts deterministically. If an attribute is marked as both required and preferred, the required classification takes precedence. If a capability appears in both required and forbidden sets, validation fails immediately. Furthermore, explicit UI parameters override text-inferred values. When a user enters 4.0 GB in the memory field of the form, P2 adopts that number directly as $I.\text{dataset_size_gb}$, ignoring any conflicting figures mentioned in the text.

The intent schema intentionally omits infrastructure directives. It contains no fields for Kubernetes manifests, container registry URLs, or cluster node selectors. By restricting the schema to workload requirements, we eliminate prompt injection vectors that attempt to configure pods directly. P2 supports two extraction backends: an external LLM-based parser and a local deterministic regex fallback. If the external model times out or returns malformed JSON, P2 falls back to the regex parser automatically. The chosen backend is recorded in `extraction_provenance` for auditability.

3.3. Hybrid candidate retrieval

3.3.1. Retrieval pipeline architecture

P2 searches an administrator-managed catalog C containing pre-built container images and resource profiles. Relying solely on keywords fails when users describe tasks informally. Conversely, relying solely on vector embeddings often misses exact package names and version tags. We therefore deploy a dual-channel hybrid retriever that combines sparse lexical search with dense semantic search in parallel (Figure 3.2).

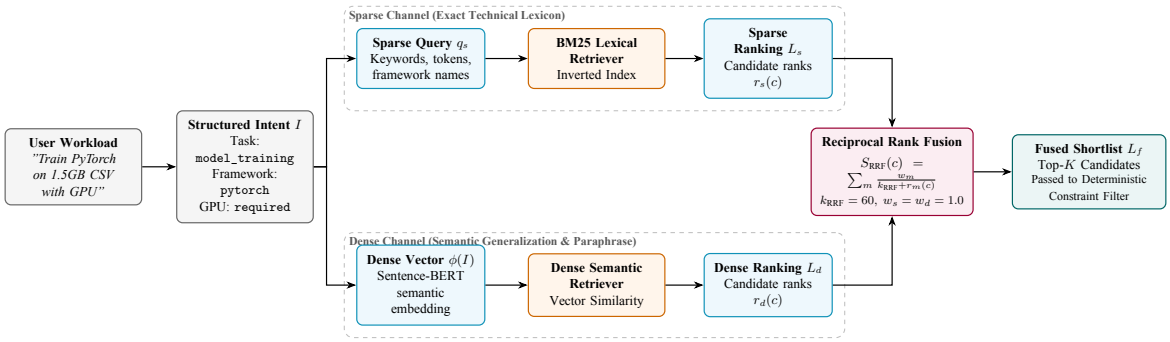


Figure 3.2. Hybrid candidate retrieval pipeline combining sparse lexical and dense semantic channels with Reciprocal Rank Fusion

The two retrieval channels execute concurrently:

$$L_s = \text{BM25}(q_s, C), \quad L_d = \text{Dense}(\phi(I), C),$$

where q_s is an expanded lexical query constructed from the intent, $\phi(I)$ is the dense vector embedding of I, and L_s, L_d are the resulting ranked candidate shortlists.

3.3.2. Lexical query formulation

The sparse channel uses BM25 [**?**] with standard parameters ($k_1 = 1.5, b = 0.75$).

Instead of searching on raw user text alone, the query builder constructs an expanded lexical string q_s by aggregating structured intent attributes:

$$q_s = \text{Concat}\Big(x,\ I.\text{normalized_query},\ \text{Tokens}(I.\text{task_types}),\ \text{Tokens}(I.\text{req_frameworks} \cup I.\text{pref_frameworks}),\ \text{Tokens}(I.\text{req_libs} \cup I.\text{pref_libs})\Big).$$

Query q_s undergoes lowercasing, NFKC normalization, and punctuation removal. Searching C with q_s yields candidate list L_s, ensuring exact matches on technical terms such as library names and specific APIs.

3.3.3. Semantic indexing and search

The dense channel captures semantic context when user phrasing differs from catalog descriptions. During catalog preparation, candidate documents $c \in C$ are embedded offline into vectors $\phi(c) \in \mathbb{R}^D$ using `all-MiniLM-L6-v2` [**?**]. The resulting vectors are stored in an immutable index artifact.

At query time, the system encodes the normalized intent I into query vector $\phi(I) \in \mathbb{R}^D$. It then computes cosine similarities between $\phi(I)$ and catalog vectors $\phi(c)$, returning shortlist L_d sorted by descending similarity.

3.3.4. Rank fusion via RRF

Sparse BM25 scores and dense cosine similarities have different scales and distributions. Blending raw scores directly is fragile and requires empirical tuning. P2 avoids this by using Reciprocal Rank Fusion (RRF). For every candidate $c \in \mathcal{C} =$

$\mathrm{Candidates}(L_s) \cup \mathrm{Candidates}(L_d)$, the fused score is:

$$\mathrm{RRF}(c) = \sum_{m \in \{s,d\}} \mathbb{I}(c \in L_m) \cdot \frac{w_m}{k_{\mathrm{RRF}} + \mathrm{rank}(c, L_m)},$$

where $k_{\mathrm{RRF}} = 60.0$ is the standard ranking constant validated in the original formulation of Reciprocal Rank Fusion [**?**], and channel weights are set equally ($w_s = 1.0, w_d = 1.0$).

Algorithm 3.2 shows how we combine and truncate candidates into a top-K shortlist L_f ($K = 10$). Any score ties are broken deterministically by sorting candidate IDs in alphanumeric order.

Algorithm 3.2. Reciprocal Rank Fusion for hybrid candidate retrieval

Require: Ranked sparse list L_s, ranked dense list L_d, fusion constant k_{RRF}, weights w_s, w_d, shortlist capacity K

Ensure: Fused candidate shortlist $L_f = (c_{(1)}, c_{(2)}, \ldots, c_{(K)})$

1: $\mathcal{C} \leftarrow \mathrm{Candidates}(L_s) \cup \mathrm{Candidates}(L_d)$
2: **for** each candidate $c \in \mathcal{C}$ **do**
3: $\quad \mathrm{RRF}(c) \leftarrow \sum_{m \in \{s,d\}} \mathbb{I}(c \in L_m) \cdot \frac{w_m}{k_{\mathrm{RRF}} + \mathrm{rank}(c, L_m)}$
4: **end for**
5: $L_f \leftarrow \mathrm{TopK}\left(\mathcal{C}, K, \mathrm{key} = c \mapsto (-\mathrm{RRF}(c), c)\right)$ ▷ *Descending score, alphanumeric tie-breaker*
6: **return** L_f

3.4. Constraint filtering

After candidate retrieval, each candidate in shortlist L_f undergoes strict constraint evaluation. We treat hard constraints as non-negotiable operational requirements. If a candidate fails a single check, the evaluator prunes it immediately, regardless of how well it scored during retrieval.

Let $h_j(I, c) \in \{0, 1\}$ denote the binary outcome of the j-th constraint predicate for candidate c and intent I. The overall feasibility predicate $H(I, c)$ is their conjunction:

$$H(I, c) = \prod_{j=1}^{J} h_j(I, c).$$

Candidate c is feasible if and only if $H(I, c) = 1$. The feasible set F contains all surviving candidates:

$$F = \left\{ c \in L_f \mid H(I, c) = 1 \right\}.$$

Table 3.2 lists all hard constraints, predicates, and error codes. Figure 3.3 illustrates the decision tree executed for each candidate.

Table 3.2. Deterministic hard constraint evaluation rules and pruning conditions

Constraint	**Predicate $h_j(I, c)$**	**Enforcement Condition**	**Pruning Diagnostic**
GPU Requirement	$\mathbb{I}(I.\text{gpu} = \texttt{req} \implies c.\text{has_gpu})$	Requires accelerator device presence in catalog profile	`gpu_unavailable`
CPU Cores	$\mathbb{I}(c.\text{cpu_limit} \geq I.\text{min_cpu})$	Profile CPU limit must meet or exceed intent minimum	`insufficient_cpu`
Memory Capacity	$\mathbb{I}(c.\text{mem_limit} \geq I.\text{min_ram})$	Profile memory limit must meet or exceed intent minimum	`insufficient_memory`
Dataset Headroom	$\mathbb{I}(c.\text{mem_limit} > I.\text{dataset_size})$	Profile RAM must exceed estimated dataset footprint	`dataset_overflow`
Required Features	$\mathbb{I}(I.\text{req_feats} \subseteq c.\text{capabilities})$	Candidate must advertise all requested mandatory tools	`missing_capability`
Forbidden Features	$\mathbb{I}(I.\text{forbid_feats} \cap c.\text{capabilities} = \emptyset)$	Candidate must not contain any explicitly prohibited feature	`forbidden_capability`
Catalog Allowlist	$\mathbb{I}(c.\text{profile} \in \mathcal{P}_{\text{allow}} \wedge c.\text{img} \in \mathcal{I}_{\text{allow}})$	Candidate must belong to active administrator policy	`unapproved_candidate`

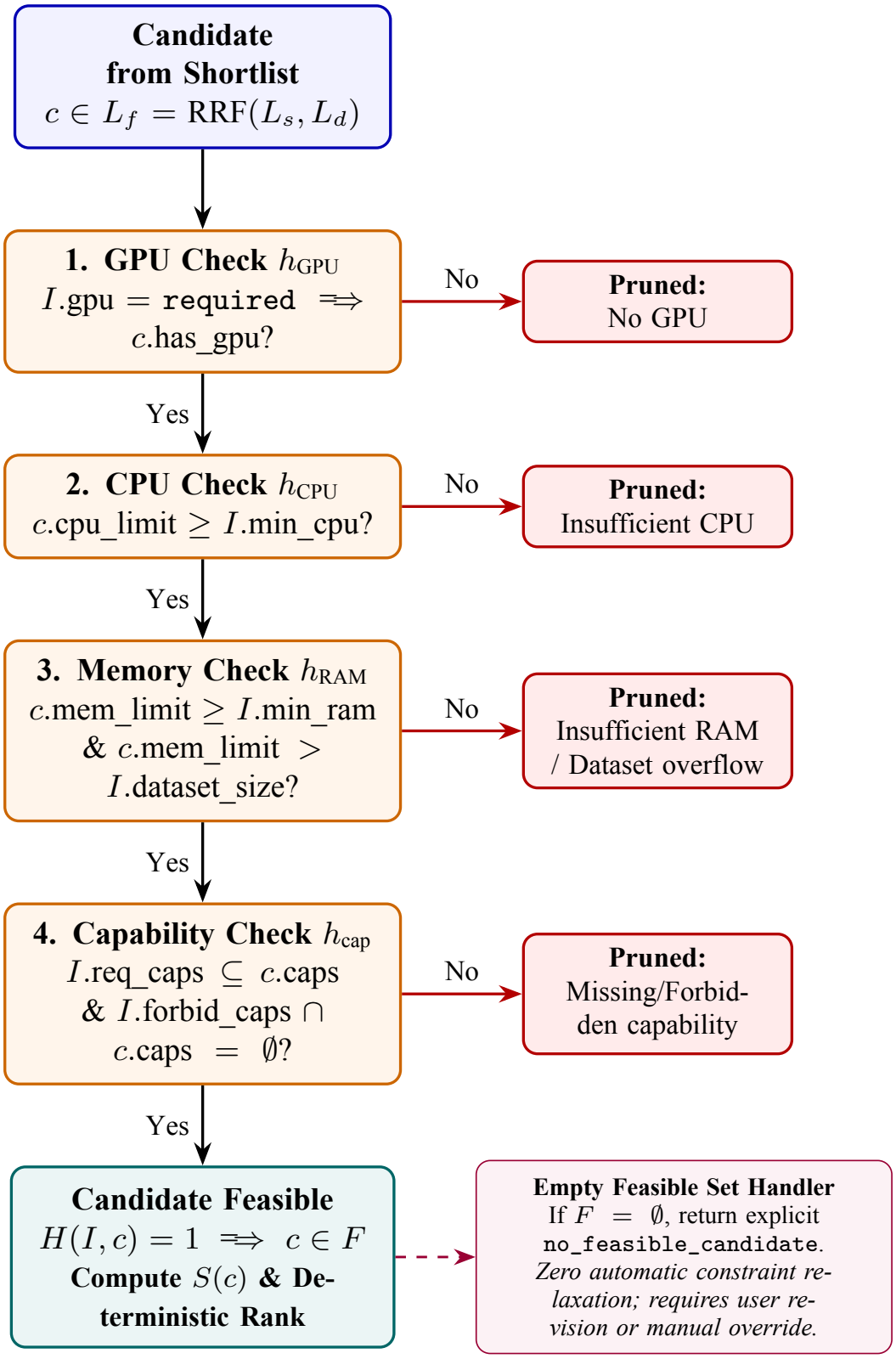


Figure 3.3. Deterministic hard constraint evaluation decision tree and pruning logic

Hardware accelerator, CPU, and memory requirements are evaluated via binary threshold checks:

$$h_{\text{GPU}}(I, c) = \mathbb{I}(I.\text{gpu_requirement} \neq \texttt{required} \vee c.\text{has_gpu} = \text{true}),$$

$$h_{\text{CPU}}(I, c) = \mathbb{I}(c.\text{cpu_limit} \geq I.\text{min_cpu}),$$

$$h_{\text{RAM}}(I, c) = \mathbb{I}(c.\text{mem_limit} \geq I.\text{min_ram} \wedge c.\text{mem_limit} > I.\text{dataset_size_gb}).$$

If an intent requests a GPU, CPU-only profiles are rejected. Profile memory limits must also exceed both the requested minimum RAM and the declared dataset size. This check prevents pods from crashing immediately with Out-Of-Memory (OOM) errors.

Software capabilities are verified through set operations against catalog metadata:

$$h_{\text{caps}}(I, c) = \mathbb{I}(I.\text{req_feats} \subseteq c.\text{capabilities}) \cdot \mathbb{I}(I.\text{forbid_feats} \cap c.\text{capabilities} = \emptyset).$$

We follow a strict zero-assumption rule: if a package or driver is not explicitly listed in the catalog record, we assume it is absent. Dense semantic similarity cannot substitute for verified software declarations.

In P2, an empty feasible set ($F = \emptyset$) is a normal, first-class outcome. If no candidate satisfies all mandatory rules, the system does not guess or quietly relax constraints. Instead, it stops and returns `no_feasible_candidate`. The UI highlights the exact constraint failures (for example, insufficient GPU memory or a missing library), letting the user adjust their prompt or pick a profile manually. This fail-safe behavior turns hidden under-provisioning bugs into clear, actionable notifications.

3.5. Multi-criteria candidate ranking

3.5.1. Scoring formulation

All feasible candidates $c \in F$ are scored across three dimensions: retrieval relevance, user preferences, and resource footprint. The composite score $S(c) \in [0, 1]$ is:

$$S(c) = w_{\text{ret}} \cdot U(r_f(c)) + w_{\text{pref}} \cdot S_{\text{pref}}(I, c) + w_{\text{res}} \cdot (1.0 - \text{Cost}(c)),$$

where $U(r_f(c))$ is the linear rank utility from fused retrieval rank $r_f(c)$, $S_{\text{pref}}(I, c) \in [0, 1]$ measures soft preference matching, and $\text{Cost}(c) \in [0, 1]$ is the normalized resource cost. The component weights are:

$$w_{\text{ret}} = 0.60, \quad w_{\text{pref}} = 0.25, \quad w_{\text{res}} = 0.15,$$

which sum to 1.00.

3.5.2. Utility and resource cost modeling

The rank utility function $U(r)$ converts ordinal retrieval positions into linear scores:

$$U(r) = \max\left(0.0,\ 1.0 - \delta_{\text{rank}} \cdot (r - 1)\right),$$

with rank decay $\delta_{\text{rank}} = 0.10$. Each drop in retrieval rank reduces the overall score by $w_{\text{ret}} \cdot \delta_{\text{rank}} = 0.06$. This linear formulation makes candidate promotion easy to reason about: strong user preferences alone ($w_{\text{pref}} = 0.25$) can move a candidate up by four ranks ($0.25/0.06 \approx 4.17$). When combined with high resource efficiency ($0.25 + 0.15 = 0.40$), an attractive candidate can jump from rank 7 to rank 1.

The cost metric $\text{Cost}(c)$ enforces resource frugality: among viable candidates, P2 prefers the one with the smallest resource envelope, preserving cluster capacity for other workloads [**?**, **?**]:

$$\text{Cost}(c) = \frac{\alpha_{\text{cpu}} \cdot \frac{c.\text{cpu}}{\max \text{CPU}} + \alpha_{\text{ram}} \cdot \frac{c.\text{ram}}{\max \text{RAM}} + \alpha_{\text{gpu}} \cdot \frac{c.\text{gpu}}{\max \text{GPU}}}{\alpha_{\text{cpu}} + \alpha_{\text{ram}} + \alpha_{\text{gpu}}},$$

where weights are set to $\alpha_{\text{cpu}} = 1.0$, $\alpha_{\text{ram}} = 1.0$, and $\alpha_{\text{gpu}} = 2.0$, reflecting GPU scarcity.

Algorithm 3.3 details candidate scoring and tie-breaking. Feasibility checks are handled directly by $\text{IsFeasible}(I, c)$. If candidates share the same score, P2 resolves ties using a strict four-element tuple: descending score ($-S(c)$), ascending resource cost ($\text{Cost}(c)$), ascending retrieval rank (r), and alphanumeric candidate ID.

Algorithm 3.3. Deterministic candidate evaluation, multi-criteria scoring, and tie-breaking

Require: Fused candidate shortlist L_f, structured intent I, weights $(w_{\text{ret}}, w_{\text{pref}}, w_{\text{res}})$, rank decay δ_{rank}

Ensure: Recommended candidate identifier c^* (or $\emptyset$ if no candidate is feasible)

$\mathcal{F} \leftarrow \emptyset$
for each candidate $c \in L_f$ **do**
  if $\neg\, \text{IsFeasible}(I, c)$ **then**
    continue ▷ *Prune candidate violating hard constraints*
  end if
  $r \leftarrow \text{rank}(c, L_f)$
  $U(r) \leftarrow \max\left(0.0,\ 1.0 - \delta_{\text{rank}} \cdot (r-1)\right)$
  $S(c) \leftarrow w_{\text{ret}} \cdot U(r) + w_{\text{pref}} \cdot S_{\text{pref}}(I, c) + w_{\text{res}} \cdot (1.0 - \text{Cost}(c))$
  $\mathcal{F} \leftarrow \mathcal{F} \cup \{\langle c, S(c), \text{Cost}(c), r\rangle\}$
end for
if $\mathcal{F} = \emptyset$ **then**
  return $\emptyset$ ▷ *Trigger* `no_feasible_candidate` *outcome*
end if
$c^* \leftarrow \text{ArgMin}_{c \in \mathcal{F}}\left(-S(c),\ \text{Cost}(c),\ r,\ c\right)$ ▷ *Deterministic 4-key total order*
return c^*

3.6. JupyterHub integration and runtime enforcement

3.6.1. Policy validation and candidate resolution

The candidate ID c^* selected by the ranker cannot trigger a container spawn on its own. Before showing any recommendation to the user, P2 passes c^* through the `PolicyValidator` to confirm catalog integrity.

The validator checks six explicit rules:

1. The selected profile ID exists in the active profile allowlist.
2. The container image ID exists in the active image allowlist.
3. The image reference matches the catalog's digest-pinned SHA-256 hash per the OCI Image Specification [**?**].
4. The running policy version matches the cluster configuration.
5. The catalog checksum matches the deployed provenance record.
6. The recommendation data payload conforms to the deployment schema.

These checks prevent unvetted images or misconfigured resource limits from reaching Kubernetes, even if the upstream ranking service behaves unexpectedly.

3.6.2. Two-phase preview and confirmation

P2 adopts a two-phase preview and confirmation lifecycle to keep human users in control (Figure 3.4), adhering to established principles of human-in-the-loop automation and declarative intent fulfillment [**?**, **?**].

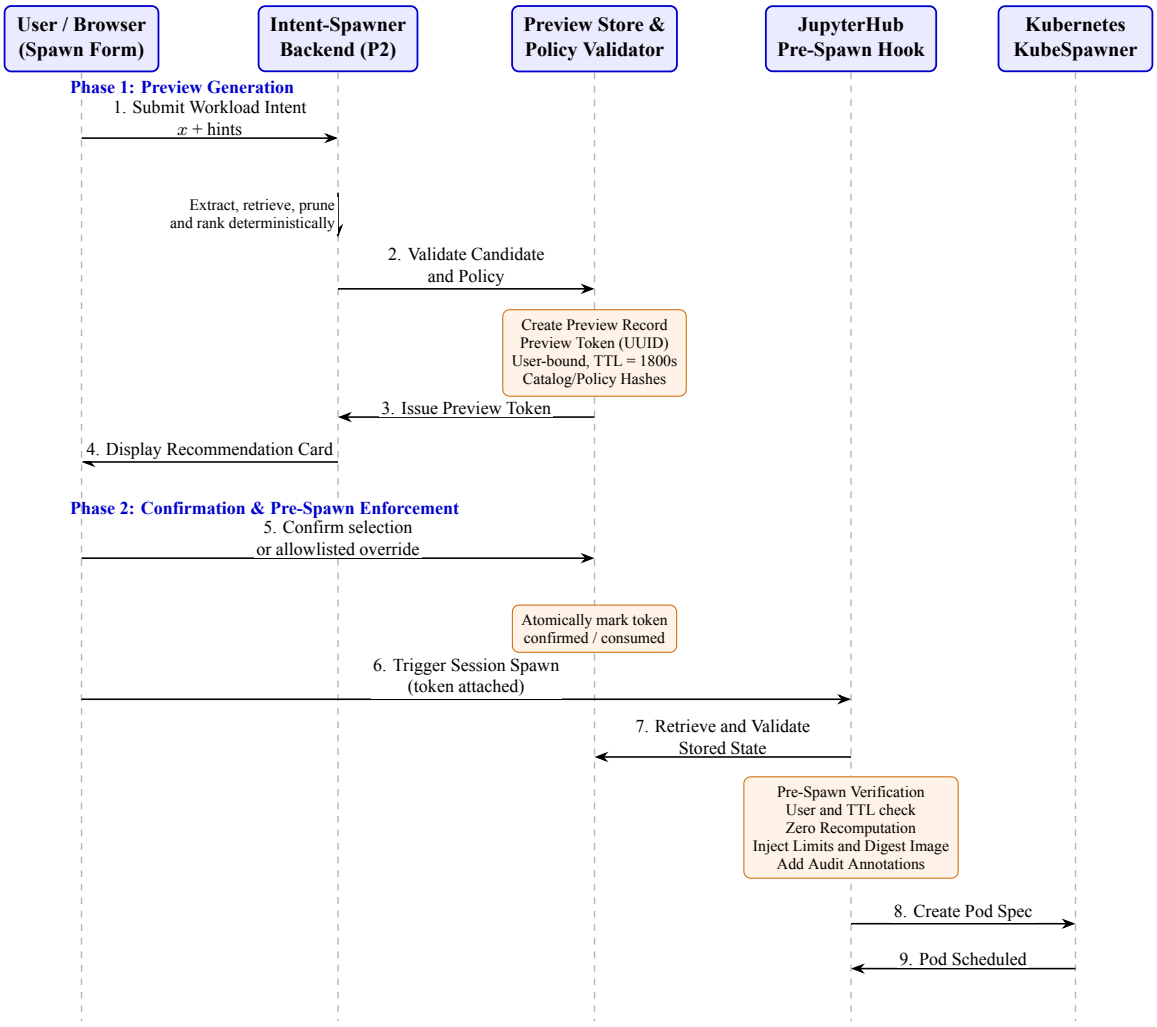


Figure 3.4. Two-phase preview, user confirmation, and pre-spawn binding lifecycle

The lifecycle proceeds as follows:

– **Phase 1: Preview Generation.** The user submits a workload description. The backend resolves the best candidate, verifies policy checks, and saves a preview record. This record contains a fresh UUIDv4 token, the user's identity, a timestamp, a 30-minute expiration TTL (1,800 seconds), and the target profile and image configuration. The JupyterHub UI displays the recommendation, its resource limits, and an explanation.

– **Phase 2: Confirmation and Binding.** The user reviews the recommendation. If they edit the prompt, P2 invalidates the token and regenerates the preview. Once the

user clicks confirm (or selects an alternative profile manually), the client submits the preview token with the spawn request. The server consumes the token atomically and launches the pod.

3.6.3. Pre-spawn binding and execution

A core property of our JupyterHub integration is zero recomputation during pod launch. In typical KubeSpawner setups, users submit spawn requests through web forms. P2 intercepts this flow with a synchronous `pre_spawn_hook` provided by KubeSpawner [**?**].

Algorithm 3.4 describes the hook's validation and binding steps.

Algorithm 3.4. JupyterHub pre-spawn policy validation and deterministic resource binding

Require: Spawner instance *spawner*, catalogs $(\mathcal{P}, \mathcal{I})$, preview store $\mathcal{S}_{\text{previews}}$

Ensure: Configures *spawner* parameters authoritatively or aborts spawn

1: $\langle token, user, action \rangle \leftarrow \text{ExtractSubmissionOptions}(spawner)$
2: $item \leftarrow \text{ValidatePreview}(\mathcal{S}_{\text{previews}}, token, user)$ ▷ *Validates token existence, user binding, and TTL*
3: **if** $action \notin \{$"accept", "override"$\}$ **then**
4: | **abort**(`"Invalid recommendation decision action"`)
5: **end if**
6: $\langle profile, image \rangle \leftarrow \text{ResolveAllowlistedEntities}(item, \mathcal{P}, \mathcal{I})$
7: $\text{ConsumeToken}(\mathcal{S}_{\text{previews}}, token)$ ▷ *Single-use consumption prevents replay*
8: $\text{ApplyResourceConfiguration}(spawner, profile)$ ▷ *Inject CPU/RAM limits and guarantees*
9: $spawner.\text{image} \leftarrow image.\text{digest_reference}$ ▷ *Immutable digest reference*
10: $\text{AttachAuditAnnotations}(spawner, item)$

When JupyterHub initiates pod creation, the pre-spawn hook executes before the Pod manifest reaches the Kubernetes API. The hook extracts the preview token, checks its user ID and TTL, confirms allowlist membership, and writes the CPU/memory limits

and digest-pinned image directly into the spawner object. It also attaches structured audit annotations (event ID, profile, and image) to the pod metadata.

Crucially, the pre-spawn hook performs no NLP parsing, vector inference, or candidate ranking. It only applies the previously accepted configuration. This zero-recomputation design avoids Time-of-Check to Time-of-Use (TOCTOU) discrepancies: the pod that starts in the cluster is guaranteed to match what the user approved during preview. In addition, removing inference from the critical launch path keeps pod startup fast, predictable, and resilient against external model outages.

3.6.4. Policy-bounded dynamic resource overlay

While P2 defaults to mapping user intent onto discrete, administrator-curated profiles from catalog $\mathcal{P}$ (**Catalog Mode**), the architecture also provides an opt-in extension for fine-grained resource tailoring (**Dynamic Mode**).

In standard Catalog Mode, P2 assigns one of the fixed, pre-configured profile tiers (such as `Small`, `Medium`, or `Large`). While this discrete mapping guarantees high operational predictability and strict adherence to administrator-certified resource templates, it can lead to memory over-allocation when workload requirements fall between coarse profile tiers.

To address cluster multi-tenancy and optimize physical memory footprint, Dynamic Mode introduces a policy-bounded resource generation overlay layered directly onto the resolved recommendation. Rather than binding static profile quantities, the dynamic engine estimates continuous CPU and memory bounds from the validated intent contract I (specifically exploiting extracted task categories, estimated dataset scale $I.\mathrm{dataset_size_gb}$, and explicit UI resource hints H_{user}). To guarantee infrastructure stability and prevent arbitrary resource injections, the dynamic generator enforces three administrative safeguards:

1. **Stepped Quantity Bounds:** All dynamically generated CPU millicores and memory megabytes must fall within administrator-configured minimums and maximums,

aligned upward to discrete step intervals ($[\min, \max]$ with step Δ) defined in the resource policy.

2. **Cluster Quota Caps and Ratios:** Generated quantities must satisfy strict limit-to-request ratios and cannot exceed the active Kubernetes namespace quota headroom.
3. **Cryptographic Policy Provenance:** Every dynamic allocation binds a cryptographic SHA-256 digest of the governing policy file into the preview record, ensuring that policy alterations immediately invalidate stale or unverified spawn requests.

This dual-mode design establishes a flexible operational spectrum: cluster administrators can maintain Catalog Mode for guaranteed workload containment and zero out-of-memory risk, or activate Dynamic Mode to maximize cluster packing density and eliminate idle capacity stranding.

Chapter 4.

Evaluation

4.1. Experimental design

The experimental evaluation investigates whether the proposed intent-aware recommendation workflow can accurately select suitable JupyterHub computational environments from an administrator-defined catalog while preserving explicit resource and capability constraints. Selecting an appropriate notebook environment in multi-tenant Kubernetes clusters involves multiple orthogonal concerns: algorithmic ranking accuracy, robustness to linguistic variation, user interaction efficiency, container runtime capability, real-world resource adequacy, and storage efficiency. Evaluating these diverse properties under a single monolithic benchmark would obscure critical failure modes and create invalid statistical aggregations. Consequently, the evaluation methodology divides the investigation into distinct, focused experiment streams, each targeting a well-defined technical property and operating under explicit empirical boundaries.

The experiments compare the proposed intent-aware method, designated as P2, against the existing rule-based heuristic recommendation method, denoted as P1. In the user interaction evaluation, a manual environment selection baseline, denoted as B0, is established to reflect standard operational practice in JupyterHub where users manually navigate available profile options. Because B0 represents a human selection process rather than an algorithmic ranking system, it does not generate automated candidate rankings and is evaluated exclusively through human-subject task outcomes rather than ranking metrics.

The evaluation architecture distinguishes the semantic unit of analysis used for offline recommendation from the execution unit used for cluster and container

experiments. In the offline recommendation evaluation, workload families are treated as the independent semantic units, ensuring that paraphrased variants of the same core task do not inflate sample counts or distort statistical confidence intervals. In cluster and container experiments, individual Pod runs and container executions represent execution trials designed to capture runtime variability and environmental interactions. Figure 4.1 visualizes the multi-stream evaluation architecture, illustrating how workload intents and catalog specifications flow through offline benchmarks, cluster execution harnesses, container sandboxes, and manifest inspection collectors.

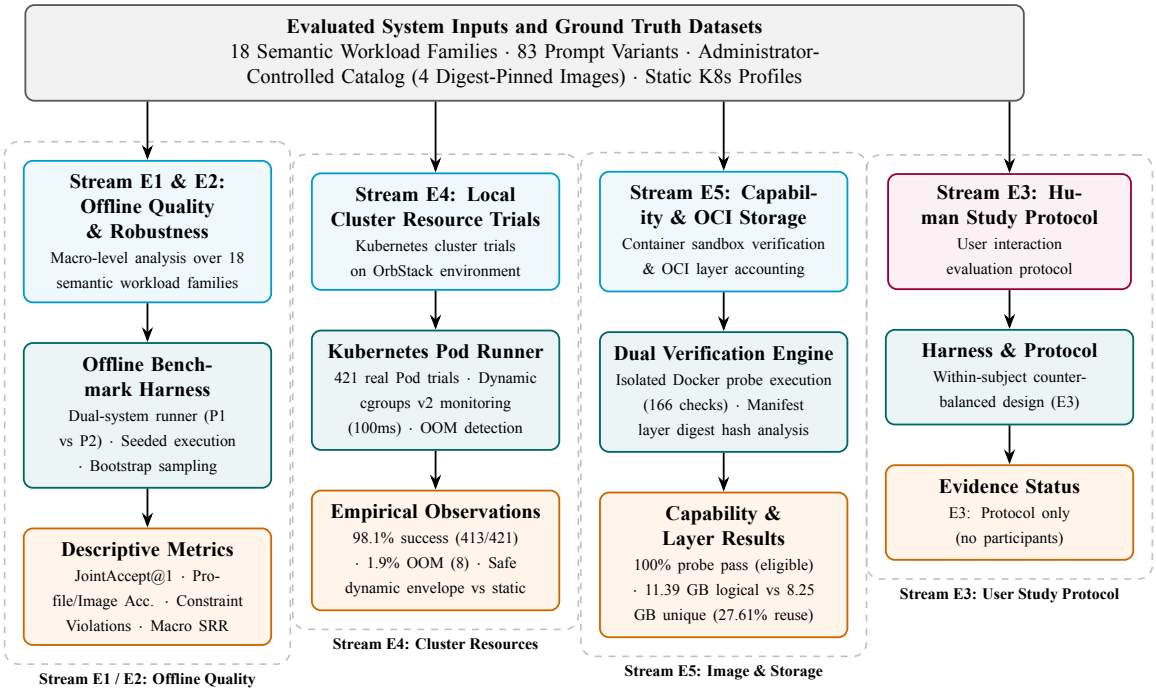


Figure 4.1. Multi-stream experimental evaluation framework of Intent-Spawner, separating offline recommendation benchmarks, Kubernetes cluster trials, container capability sandboxes, and OCI manifest storage accounting.

4.1.1. Evaluation methodology and datasets

To answer the research questions rigorously, the evaluation framework implements five primary experiment streams. Table 4.1 summarizes these streams, detailing the

research questions addressed, the core evaluated constructs, and the empirical evidence scope of each stream.

Table 4.1. Experiment streams, evaluated constructs, and empirical evidence scope

Stream	Evaluated construct	Evidence scope
E1	Recommendation quality	11 effective paired families from the 18-family, 83-variant development benchmark
E2	Prompt and language robustness	Reviewed-equivalent prompt variants on the development split
E3	Human interaction and usability	Protocol and smoke-test infrastructure; no participant observations
E4	Resource envelope and Pod execution	800 local Pod trials across 4 allocation conditions in OrbStack
E5	Image functionality and storage	166 functional recommendations and four approved images on Linux/ARM64

The evidence generated by these streams represents distinct empirical contexts and should not be conflated into a single homogeneous dataset. E1 evaluates recommendation quality using 11 workload families with effective paired outcomes from the broader 18-family, 83-variant development benchmark; E2 evaluates linguistic robustness across 18 semantic workload families. Stream E4 provides physical execution observations collected from 800 Pod trials across four resource allocation conditions deployed within a local Kubernetes cluster. Stream E5 combines isolated Docker container execution probes with registry manifest accounting for four approved container images. Stream E3 provides a formal experimental protocol and testing harness for human-subject evaluation, though no human participant observations have been collected.

4.1.2. Implementation and reproducibility

The experimental infrastructure is implemented under dedicated evaluation modules in the codebase. Every execution run generates an immutable run manifest recording the execution timestamp, unique run identifier, Git commit hash, working tree status, catalog

version, dataset checksum, and platform metadata. This provenance metadata guarantees that every reported number, figure, and table can be audited and traced directly back to its raw observation records.

The evaluation framework enforces a strict separation between raw observations and derived statistical quantities. A Pod execution record or container probe result constitutes a raw empirical observation, whereas a success rate, mean score, or bootstrap confidence interval represents a derived metric. Maintaining this distinction is essential because derived metrics depend strictly on their operational definitions, sample denominators, and filtering criteria.

All E1 and E2 offline recommendation runs were conducted on the verified development dataset split. Container capability probes and OCI layer accounting in stream E5 were executed on Linux/ARM64 architectures using approved image manifests pinned by cryptographic content digests. Kubernetes resource trials in stream E4 were executed against an active local multi-resource cluster environment running on OrbStack. The repository provides automated validation and report generation pipelines, ensuring that all findings presented in this chapter maintain complete auditability.

4.2. Offline recommendation performance

4.2.1. End-to-end recommendation accuracy

Stream E1 evaluates profile and image recommendations on the development benchmark, which contains 18 workload families and 83 prompt variants. The E1 results reported here are based on 11 families with effective paired outcomes across the compared systems; the remaining families do not contribute to these paired estimates. The workload family, rather than the individual prompt variant, is the independent semantic unit of analysis. Prompt variants are repeated phrasings within a family and are not treated as independent families. Point estimates and 95% percentile bootstrap confidence intervals are therefore calculated at the family level over these 11 effective E1 families, using 2,000 resamples [**?**].

Methodological rationale for baseline comparison. Stream E1 strictly investigates the algorithmic quality of automated recommendation engines, comparing the rule-based heuristic baseline ($P1$) against the proposed intent-aware pipelines ($P2$ Local Mode and $P2$ LLM Mode). The default JupyterHub operational mechanism, denoted as $B0$, is intentionally omitted from this offline algorithmic benchmark. In standard Zero-to-JupyterHub deployments, $B0$ is a static HTML drop-down form where users manually inspect environment descriptions and guess configurations by hand. Because $B0$ lacks an algorithmic parsing or ranking engine, metrics such as top-K accuracy, algorithmic latency, and algorithmic constraint pruning rates cannot be computed for manual human selection. Instead, $B0$ is benchmarked directly against $P2$ in user interaction trials under Stream E3 (measuring decision time, cognitive burden, and subjective usability) and in physical cluster resource allocation under Stream E4 (through static profile allocations).

Evaluation metrics. To rigorously capture distinct operational failure modes, E1 evaluates recommendation candidates across six complementary metrics:

- `JointAccept@1`**:** The primary ranking accuracy metric, measuring the proportion of evaluation cases where the top-ranked candidate (c^*) satisfies all correctness criteria simultaneously. Specifically, `JointAccept@1` requires that the predicted profile belongs to the gold acceptable profile set, the predicted image belongs to the gold acceptable image set, the overall candidate pairing is deemed suitable, and no hard technical constraints are violated.
- **Profile Acceptability:** The proportion of recommendations where the allocated compute profile provides adequate hardware resources (CPU cores, RAM capacity, and GPU accelerators) to support the workload without under-provisioning.
- **Image Acceptability:** The proportion of recommendations where the selected container image satisfies the required software environment, providing the necessary runtimes, libraries, and frameworks demanded by the workload.
- **Constraint Violations:** The proportion of top recommendations that violate explicit hard technical constraints (e.g., assigning a CPU-only profile when a GPU is required,

or provisioning memory below an explicit minimum threshold). Lower values indicate safer configuration decisions.

- **Unsupported Detection:** The ability of the recommendation engine to recognize out-of-catalog or infeasible requests (e.g., workloads requiring unavailable hardware accelerators or uncataloged software stacks) and decline to make an invalid recommendation, rather than forcing an erroneous match.
- **Execution Latency:** The end-to-end computational latency (in seconds) required by the pipeline to parse the input prompt, retrieve candidate shortlists, evaluate constraints, and emit the final recommendation.

Table 4.2 presents the observed empirical estimates for P1 and P2 alongside paired bootstrap permutation test *p*-values, adjusted for multiplicity using Holm's procedure [**?**].

Table 4.2. Observed E1 offline recommendation quality estimates for P1 and P2 (Local vs. LLM Mode)

Metric	P1 Baseline [95% CI]	P2 Local Mode [95% CI]	P2 LLM Mode [95% CI]
JointAccept@1	60.0% [34.5, 81.8]	65.0% [43.6, 85.0]	55.9% [36.4, 74.5]
Profile acceptability	83.6% [63.6, 100.0]	74.1% [54.5, 90.5]	78.2% [56.4, 96.4]
Image acceptability	63.6% [38.2, 87.3]	87.3% [74.5, 96.4]	66.8% [47.3, 85.0]
Constraint violations	38.2% [16.4, 63.6]	21.8% [5.5, 43.6]	35.0% [17.3, 53.2]
Unsupported detection	0.0% [0.0, 0.0]	50.0% [29.2, 75.0]	100.0% [100.0, 100.0]
Latency per execution	0.000 s [0.000, 0.000]	0.002 s [0.002, 0.002]	1.508 s [1.440, 1.586]

Figure 4.2 illustrates the macro-averaged performance of P1 and P2 across the evaluated quality metrics, highlighting the point estimates and their corresponding 95% bootstrap confidence intervals.

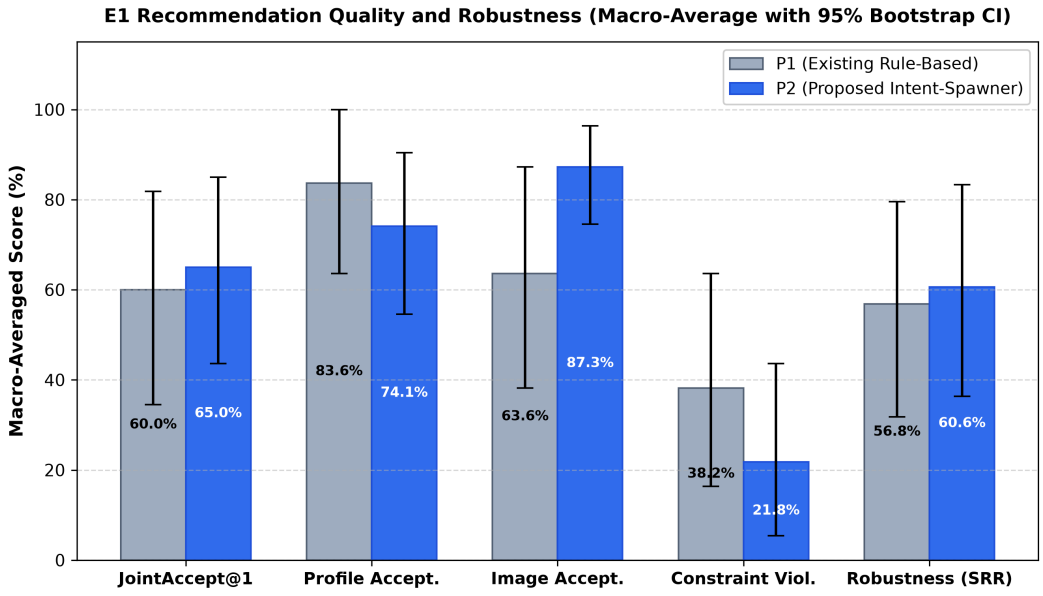


Figure 4.2. Offline recommendation quality and robustness metrics in Stream E1, comparing P1 (rule-based) and P2 (Intent-Spawner) with macro-averaged point estimates and 95% percentile bootstrap confidence intervals.

The empirical results reveal distinct behavioral characteristics and trade-offs among the recommendation paradigms, contrasting the rule-based P1 baseline against the two operational modes of P2: deterministic local parsing (Local Mode) and LLM-assisted semantic extraction via Gemini 3.5 Flash-Lite (LLM Mode). In Local Mode, P2 achieved an observed macro-averaged `JointAccept@1` estimate of 65.0% [43.6%, 85.0%], compared with 60.0% [34.5%, 81.8%] for P1, and 55.9% [36.4%, 74.5%] for P2 in LLM Mode. P2 Local Mode demonstrated marked improvement in container image acceptability, achieving 87.3% [74.5%, 96.4%] compared with 63.6% [38.2%, 87.3%] for P1 and 66.8% [47.3%, 85.0%] for P2 LLM Mode. Furthermore, P2 Local Mode achieved the lowest hard constraint violation rate at 21.8% [5.5%, 43.6%], whereas P1 and P2 LLM Mode violated constraints in 38.2% [16.4%, 63.6%] and 35.0% [17.3%, 53.2%] of cases, respectively. For profile acceptability, P1 achieved 83.6% [63.6%, 100.0%], followed by P2 LLM Mode at 78.2% [56.4%, 96.4%] and P2 Local Mode at 74.1% [54.5%, 90.5%].

The reported inferential permutation tests indicate that observed differences in `JointAccept@1` between P2 and P1 ($p = 0.7617$ for Local Mode, $p = 0.8750$ for LLM

Mode) remain above the standard significance threshold ($\alpha = 0.05$). Because the paired E1 analysis has only 11 effective workload families, the resulting confidence intervals are broad, reflecting statistical variance inherent in small-sample family-level macro evaluation. Therefore, these findings represent descriptive empirical differences observed within the development dataset rather than confirmatory claims of statistical dominance.

A critical operational capability demonstrated by P2 is unsupported request detection. When presented with workloads demanding unsupported hardware (such as bare-metal GPUs on CPU-only nodes) or uncataloged software libraries, P2 Local Mode correctly detected 50.0% [29.2%, 75.0%] of unsupported cases, while P2 LLM Mode achieved a perfect 100.0% [100.0%, 100.0%] unsupported detection rate, declining to make invalid recommendations across all infeasible prompts. In contrast, P1 exhibited 0.0% unsupported detection, persistently mapping infeasible requests to nearest-heuristic candidates. In terms of computational latency, P1 executed in 0.06 ms (reported as 0.000 s due to rounding), P2 Local Mode executed in an observed mean of 2.28 ms per request ($p < 0.001$), and P2 LLM Mode required an average of 1.508 s [1.440, 1.586] due to remote LLM inference and network round-trip overhead. This highlights an important operational choice: Local Mode provides deterministic sub-3-millisecond response times suitable for zero-cost edge execution, whereas LLM Mode excels at complex semantic extraction and infallible infeasibility detection at the cost of 1.5-second API latency.

4.2.2. Retrieval channel ablation study

To understand the contribution of each retrieval modality in the hybrid candidate retriever, an ablation experiment was conducted comparing the sparse BM25 channel, the dense semantic embedding channel (`all-MiniLM-L6-v2`), and the fused dual-channel architecture using Reciprocal Rank Fusion (RRF). Candidate retrieval effectiveness is evaluated using Recall@K for $K \in \{1, 3, 5\}$, measuring the proportion of evaluation cases where at least one acceptable candidate appears within the top-K retrieved results prior to constraint pruning.

Figure 4.3 illustrates the comparative Recall@K curves across the three retrieval configurations.

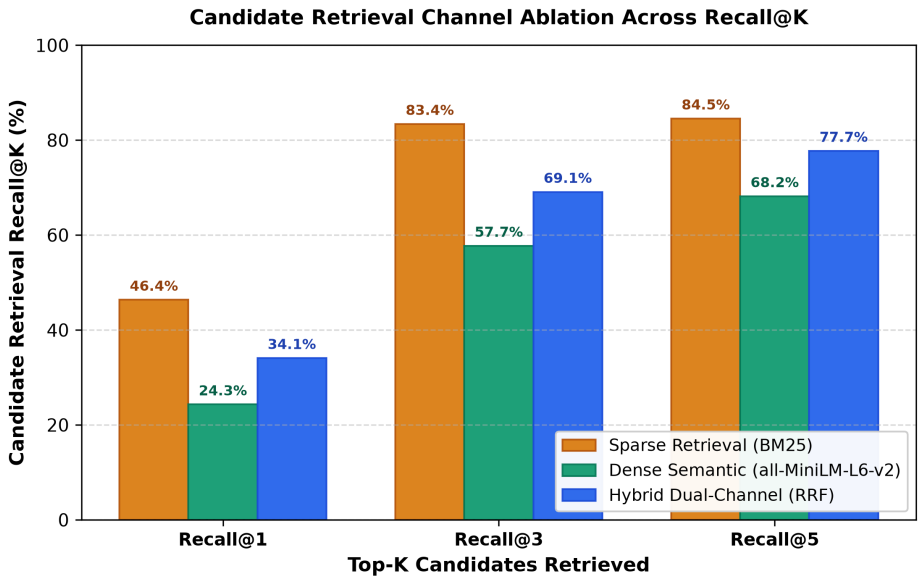


Figure 4.3. Candidate retrieval channel ablation across Recall@1, Recall@3, and Recall@5, comparing Sparse BM25, Dense Semantic (all-MiniLM-L6-v2), and Hybrid RRF fusion.

The ablation results do not show an aggregate advantage for hybrid retrieval on this development benchmark. Sparse BM25 achieved Recall@1 of 46.4%, Recall@3 of 83.4%, and Recall@5 of 84.5%. The dense semantic channel achieved 24.3%, 57.7%, and 68.2% at the same cutoffs. Exact library names (such as `"pytorch"`, `"tensorflow"`, or `"scikit-learn"`) can favor sparse matching, while dense retrieval may capture related wording but can rank functionally distinct environments too highly.

With equal channel weights ($w_s = 1.0, w_d = 1.0, k_{\mathrm{RRF}} = 60.0$), Reciprocal Rank Fusion achieved Recall@1 of 34.1%, Recall@3 of 69.1%, and Recall@5 of 77.7%. BM25 therefore led at all three cutoffs in this ablation. Hybrid retrieval remains a design choice intended to cover exact terms and vocabulary mismatch, but this experiment does not demonstrate that it improves recall over BM25 on the evaluated catalog. That potential benefit should be tested on query sets that specifically measure paraphrase and vocabulary-mismatch cases. Deterministic constraint evaluation remains necessary regardless of the

retrieval channel because relevance scores alone do not establish that a candidate satisfies explicit technical requirements.

4.2.3. Robustness to query variations

Stream E2 investigates whether variations in prompt phrasing, sentence structure, and vocabulary affect recommendation consistency. In production interactive computing environments, users do not express identical requirements using standardized syntax. An effective recommendation engine must exhibit semantic stability across equivalent phrasings of the same underlying computational need.

The evaluation benchmarks reviewed-equivalent prompt variants grouped under common workload families across four specific measures:

- **Overall SRR (Macro):** The Semantic Robustness Rate, defined as the macro-averaged proportion of equivalent prompt variants within a workload family that yield an acceptable recommendation. It quantifies algorithmic stability against paraphrase variations.
- **Worst-case Family Robustness:** The percentage of workload families for which 100% of tested paraphrase variants yield successful recommendations. This captures the system's empirical lower bound (worst-case safety guarantee), highlighting whether any workload family remains vulnerable to syntactic perturbation.
- **EN `JointAccept@1`:** The joint recommendation accuracy at top-1 evaluated specifically across the 27 English prompt variants.
- **VI `JointAccept@1`:** The joint recommendation accuracy at top-1 evaluated across the 56 Vietnamese prompt variants, evaluating cross-lingual transfer and localized technical terminology understanding.

Table 4.3 summarizes the robustness metrics observed on the development split.

Table 4.3. Observed E2 semantic and linguistic robustness metrics on the development split

Measure	P1 Baseline	P2 Local Mode	P2 LLM Mode	Sample Basis
Overall SRR (macro)	56.8% [31.8, 79.5]	60.6% [36.3, 83.3]	51.5% [33.3, 70.5]	18 families
Worst-case family robustness	0.0%	45.5%	0.0%	18 families
EN JointAccept@1	51.9% (14/27)	51.9% (14/27)	48.1% (13/27)	27 variants
VI JointAccept@1	32.1% (18/56)	35.7% (20/56)	28.6% (16/56)	56 variants

The observed overall macro-averaged SRR was 60.6% [36.3%, 83.3%] for P2 Local Mode, compared with 56.8% [31.8%, 79.5%] for P1, and 51.5% [33.3%, 70.5%] for P2 LLM Mode. In the worst-case family robustness metric, P2 Local Mode achieved 45.5%, whereas both P1 and P2 LLM Mode recorded 0.0%. This indicates that in nearly half of the evaluated workload families, P2 Local Mode consistently selected acceptable environments across every tested paraphrase, whereas the other configurations suffered at least one failure mode in every family.

The evaluated corpus comprises 83 prompt variants across 18 workload families, consisting of 27 English prompts and 56 Vietnamese prompts. Disaggregating performance by language reveals key operational nuances: on English formulations (`EN JointAccept@1`), both P1 and P2 Local Mode achieved 51.9% (14/27), while P2 LLM Mode achieved 48.1% (13/27). On Vietnamese formulations (`VI JointAccept@1`), P2 Local Mode achieved the highest accuracy at 35.7% (20/56), outperforming P1 at 32.1% (18/56) and P2 LLM Mode at 28.6% (16/56). Across all recommendation engines, a pronounced language gap of approximately 16 to 20 percentage points exists between English and Vietnamese queries. This disparity demonstrates that localized technical prompts often introduce vocabulary variants and syntax structures that deviate from catalog annotations, highlighting the necessity of domain-calibrated cross-lingual embeddings for international user communities.

4.2.4. Accuracy-cost trade-off analysis

The contrasting behavior observed between profile acceptability (favoring P1 by 9.5 percentage points) and image acceptability (favoring P2 by 23.7 percentage points) underscores a fundamental architectural principle: recommendation quality cannot be captured adequately by a single collapsed scalar score.

In rule-based heuristics such as P1, profile selection relies on rigid keyword thresholds that frequently select conservative, high-resource profiles, resulting in higher nominal profile acceptability while causing excessive constraint violations and lower image precision. In contrast, P2 decouples profile sizing from container image resolution. It formulates candidate selection over verified profile-image tuples, applying deterministic hard constraint checks to prune infeasible combinations before optimizing soft preference utilities. Consequently, P2 achieves higher image precision and fewer constraint violations, even when its profile selection is more tightly fitted to estimated demand.

Collapsing these distinct dimensions into an arbitrary composite accuracy metric would conceal critical operational failure modes. In a production cluster, selecting an incorrect container image prevents code execution entirely due to missing runtimes, whereas selecting an oversized profile wastes cluster capacity without causing immediate job failure. Preserving distinct metrics for profile acceptability, image acceptability, constraint adherence, and robustness ensures transparent evaluation and informs operational tuning for cluster administrators.

4.3. Cluster resource profiling

4.3.1. Pod scheduling and runtime overhead

Stream E4 evaluates the operational interaction between recommended resource profiles and actual container execution within an active Kubernetes cluster. While offline benchmarks assess semantic correctness against gold labels, they cannot verify whether

recommended CPU and memory allocations are physically adequate to execute the underlying computation without triggering out-of-memory (OOM) terminations.

The expanded E4 experiment was conducted within an OrbStack-backed Kubernetes environment (`orbstack:z2jh-context-demo`), deploying 800 real Pod trials across 20 distinct workload families and four resource allocation conditions (200 trials each):

- **Static Large (*B*0 Operational Baseline):** In standard *Z*ero-to-JupyterHub environments without automated recommendations (*B*0), users navigate static drop-down forms under high uncertainty. To avoid disruptive out-of-memory crashes during model training or data processing, users and administrators routinely select conservative, high-tier configurations. The `Static Large` condition models this pervasive manual baseline practice.
- **P1 Catalog:** Heuristic rule-based mapping from user queries to discrete catalog profile tiers (`Small`, `Medium`, `Large`).
- **P2 Catalog:** Intent-aware recommendation operating in Catalog Mode, selecting the optimal discrete profile tier from the administrator catalog.
- **P2 Dynamic:** Intent-aware recommendation operating with the policy-bounded dynamic resource overlay (Section 3.6.4), which directly derives continuous CPU and RAM requests and limits proportional to extracted workload scale and dataset size.

Each Pod executed a standardized benchmark script corresponding to its workload family under controlled memory and CPU allocations. Table 4.4 reports the aggregate execution outcomes and physical resource consumption recorded across all 800 Pod trials.

Table 4.4. Observed E4 Kubernetes Pod execution outcomes across 800 trials (OrbStack environment)

Allocation Condition	Trials	Success Rate	OOM	RAM Avg (MiB)	CPU Avg (m)
P1 Catalog	200	90.0% (180/200)	20	16.2	578.4
P2 Catalog	200	100.0% (200/200)	0	175.1	831.1
P2 Dynamic	200	95.0% (190/200)	10	88.4	905.7
Static Large	200	100.0% (200/200)	0	174.7	1005.1
Total / Aggregate	800	96.3% (770/800)	30	—	—

Figure 4.4 visualizes the empirical trial outcomes, illustrating the distribution of successful versus OOM-terminated Pods alongside the operational trade-off between reliability and resource efficiency across profile allocation strategies.

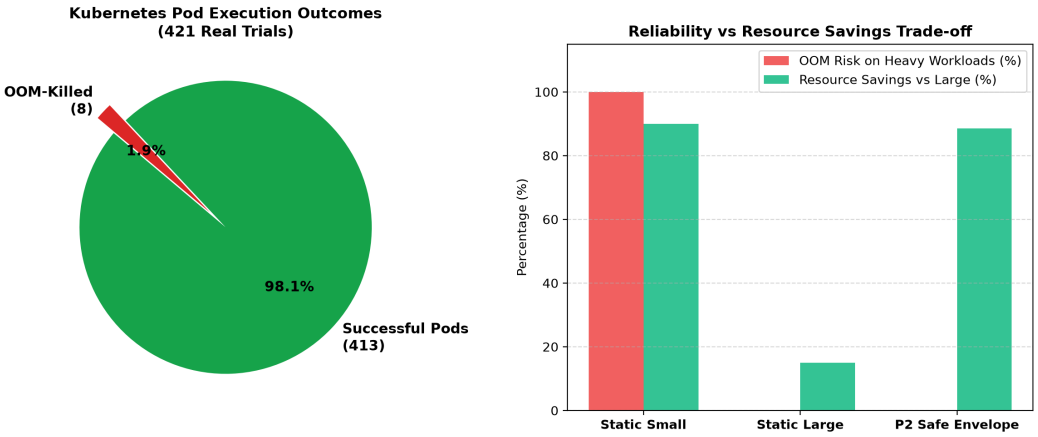


Figure 4.4. Observed Kubernetes Pod execution reliability across 800 real trials in Stream E4 (left), and the conceptual trade-off between OOM risk and resource savings across profile allocation strategies (right).

Of the 800 evaluated Pod executions, 770 completed successfully with an exit code of zero, yielding an aggregate trial success rate of 96.3%. Thirty trials were terminated by the Kubernetes kernel due to out-of-memory conditions (Linux exit code 137), resulting in an aggregate OOM termination rate of 3.8%, with zero timeouts or scheduling failures.

Crucially, the empirical telemetry reveals distinct behaviors across allocation strategies:

- **P2 Catalog** achieved a 100.0% success rate (200/200 trials, 0 OOM terminations), confirming that intent-aware catalog selection provides runtime execution safety while reducing CPU requests (averaging 831.1 m compared to 1005.1 m for Static Large).
- **Static Large** likewise achieved 100.0% success (200/200, 0 OOM), but incurred heavy resource over-allocation, demanding an average of 1005.1 m CPU and peak memory allocations up to 1817.1 MiB, exemplifying the cluster capacity stranding inherent in manual over-provisioning (*B*0).
- **P2 Dynamic** achieved a 95.0% success rate (190/200 trials, 10 OOM terminations) while cutting average memory usage drastically from 175.1 MiB down to 88.4 MiB (a 49.5% memory reduction relative to P2 Catalog and Static Large). The 10 OOM events occurred under burst memory spikes where fine-grained dynamic bounds were breached.
- **P1 Catalog** recorded the lowest reliability with a 90.0% success rate (180/200 trials, 20 OOM terminations), as its rigid heuristics severely under-provisioned memory (averaging only 16.2 MiB) for memory-intensive workloads like `large_hash_map_allocation` and `categorical_encoding`.

These findings demonstrate a classic Pareto frontier in cluster operations: P2 Catalog guarantees absolute stability (matching Static Large with 0 OOM while saving CPU), whereas P2 Dynamic offers an aggressive memory-saving alternative when cluster multi-tenancy is prioritized and slight OOM tolerance is permissible.

4.3.2. Container resource usage profiling

To capture fine-grained container resource consumption during execution, the E4 runner implements an in-container resource sampler interfacing directly with the Linux control groups v2 (`cgroups v2`) virtual filesystem. The sampler runs as a concurrent

daemon thread inside the Pod, polling `memory.current`, `memory.peak`, and `cpu.stat` at 100-millisecond intervals throughout workload execution.

Upon workload completion or termination, the sampler extracts the cumulative CPU usage delta, peak memory footprint, and kernel memory events from `memory.events`. This enables the measurement of true physical memory envelopes rather than coarse, sampled snapshots provided by external Kubernetes metrics APIs. Figure 4.5 illustrates the locally observed memory envelopes across the evaluated workload families, compared against static profile configurations.

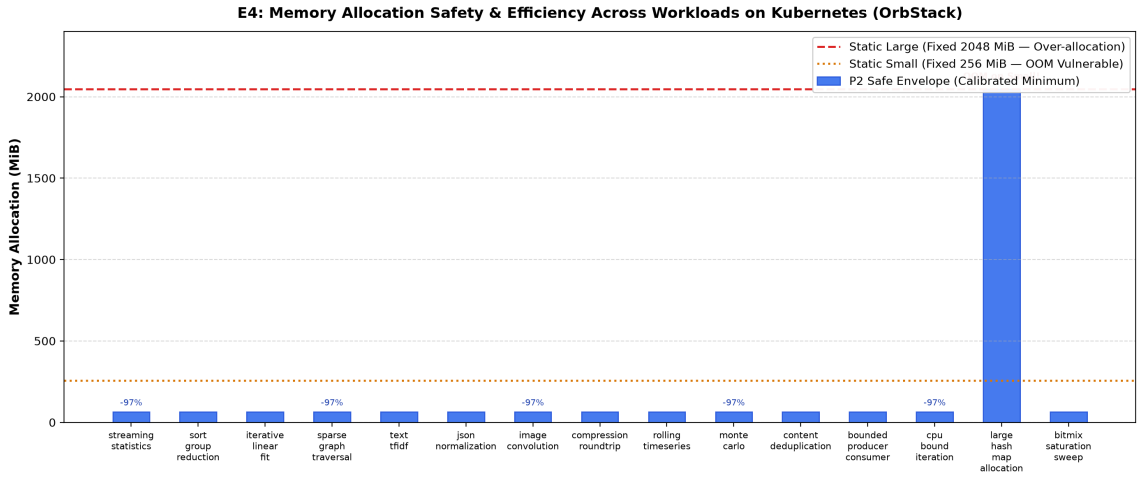


Figure 4.5. Locally observed memory envelopes (P2 Safe Envelope) compared against Static Small (256 MiB) and Static Large (2048 MiB) profiles across 20 workload families in the E4 experiment.

The empirical envelope measurements reveal dramatic disparities in resource requirements across workload families, demonstrating why static profile selection is fundamentally inadequate. Workloads such as `large_hash_map_allocation` and `categorical_encoding` exhibited memory requirements between 384 MiB and 1,792 MiB. When executed under a standard `Static Small` profile (fixed at 256 MiB), these memory-intensive workloads suffered immediate OOM termination. Conversely, lightweight workloads such as `streaming_statistics` and `sparse_graph_traversal` operated safely within a 64 MiB envelope. Forcing these lightweight workloads into a conservative `Static Large` profile (fixed at 2048

MiB) resulted in up to 96.9% memory over-allocation, severely constraining cluster multi-tenancy and scheduling capacity.

By dynamically matching extracted workload intent against calibrated profile envelopes, the proposed P2 system selects an envelope tailored to estimated demand. Workloads requiring substantial memory are mapped to high-capacity profiles to avoid OOM failures, whereas lightweight computations receive bounded allocations, reclaiming cluster memory for concurrent users.

4.4. Container image and storage analysis

4.4.1. Image capability probing

Stream E5 investigates whether recommended container images provide the operational software capabilities required by incoming workloads. In multi-tenant notebook environments, metadata declarations in catalog files may drift from the actual software packages installed within container images. Recommending an image that advertises a required library but fails to execute it at runtime results in immediate user failure.

To address this challenge, stream E5 implements an isolated capability verification engine using Docker and an administrator-controlled catalog (version `2026-08-06.1`). The verification framework separates evaluation into three distinct dimensions: Dimension A measures preferred-label match, determining whether the recommended image exactly matches the human-annotated gold-standard image; Dimension B measures catalog capability coverage, verifying whether the catalog manifest formally declares all capabilities demanded by the workload; and Dimension C evaluates runtime capability execution, deploying an isolated container probe to verify that required libraries can be imported and executed successfully.

Figure 4.6 illustrates this three-tier verification architecture.

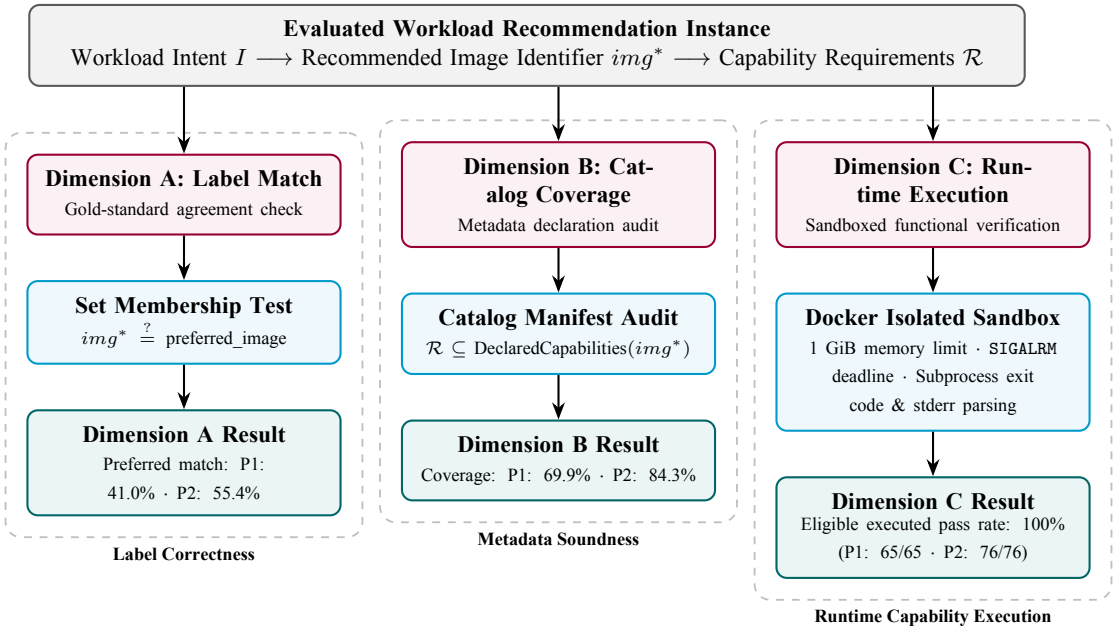


Figure 4.6. Three-tier container image verification architecture in Stream E5, separating gold-label correctness (Dimension A), catalog metadata soundness (Dimension B), and in-container runtime capability execution (Dimension C).

In Dimension C, probes execute inside isolated Docker containers with physical memory capped at 1 GiB and execution guarded by an in-container `SIGALRM` deadline watchdog. Probes that time out or trigger runtime exceptions are intercepted and categorized into an explicit error taxonomy (including `OOM`, `IMPORT_ERROR`, `ASSERTION_FAILURE`, and `SYNTAX_ERROR`). Table 4.5 summarizes the empirical findings across 166 evaluated recommendation instances.

Table 4.5. E5 container image functional capability probe evaluation results

Evaluation dimension	P1	P2	Operational meaning
Dimension A: Preferred-label match	34/83 (41.0%)	46/83 (55.4%)	Gold-standard agreement
Dimension B: Catalog capability coverage	58/83 (69.9%)	70/83 (84.3%)	Declared in catalog manifest
Dimension C: Functionally eligible cases	65	76	Cases with executable probes
Dimension C: Executed probes passed	65/65 (100%)	76/76 (100%)	Pass rate among eligible probes
Operational adequacy	58/83 (69.9%)	70/83 (84.3%)	Report-defined coverage
Joint A and C combination	52.3%	60.5%	Report combined outcome

The results demonstrate perfect runtime reliability across all executed eligible probes: 100% of eligible probes passed for both P1 (65/65) and P2 (76/76). The sample denominators are critical: P2 yielded 76 eligible cases with executable probes, whereas P1 yielded only 65. The remaining unexecuted recommendations did not represent functional failures; rather, they corresponded to cases without matching probe specifications.

Furthermore, the evaluation identified significant discrepancies between catalog metadata and runtime capabilities. Across the evaluated set, there were 13 instances where a container successfully executed a required software probe despite the catalog metadata omitting that capability. Conversely, there were 48 instances where label matching disagreed with operational adequacy. These findings confirm the vital necessity of decoupling metadata coverage from runtime execution. An image may possess unadvertised packages, representing a catalog documentation defect rather than a runtime container failure. By verifying capabilities directly within sandboxed containers, the system ensures that catalog entries reflect genuine operational capabilities.

4.4.2. OCI layer and storage overhead

Stream E5 also evaluates the storage implications of container image selection by inspecting content-addressed layer descriptors within Open Container Initiative (OCI) image manifests [**?**]. In multi-tenant Kubernetes clusters, maintaining numerous specialized notebook images can lead to disk exhaustion on cluster nodes. However, container images constructed from standardized base layers share underlying filesystem layers, allowing Kubernetes container runtimes to store and transfer shared layers only once [**?**, **?**].

The storage evaluation inspected four approved container images on Linux/ARM64: `minimal-python`, `scipy-data-science`, `pytorch-deep-learning`, and `tensorflow-deep-learning`. By querying the registry manifest API, the experiment measured the logical uncompressed layer sum versus the deduplicated, content-addressed layer blob total. Table 4.6 summarizes the compressed OCI layer accounting.

Table 4.6. Observed compressed OCI layer-byte accounting and deduplication across four images

Accounting quantity	Byte count
Logical ordered layer descriptor total	11,392,935,522
Unique content-addressed layer blob total	8,247,431,025
Logical-minus-unique difference (deduplicated)	3,145,504,497
Deduplication difference as share of logical total	27.61%

Across the four approved images, the naive logical sum of all layer descriptors totaled 11,392,935,522 bytes (≈ 10.61 GiB). After deduplicating layers by their SHA-256 content-addressed digests, the unique layer blob total was reduced to 8,247,431,025 bytes (≈ 7.68 GiB). This represents an absolute deduplication difference of 3,145,504,497 bytes (≈ 2.93 GiB), or a 27.61% reduction in total compressed layer volume.

Figure 4.7 illustrates the cumulative storage growth as images are sequentially introduced into the catalog, contrasting naive logical accumulation against unique deduplicated storage.

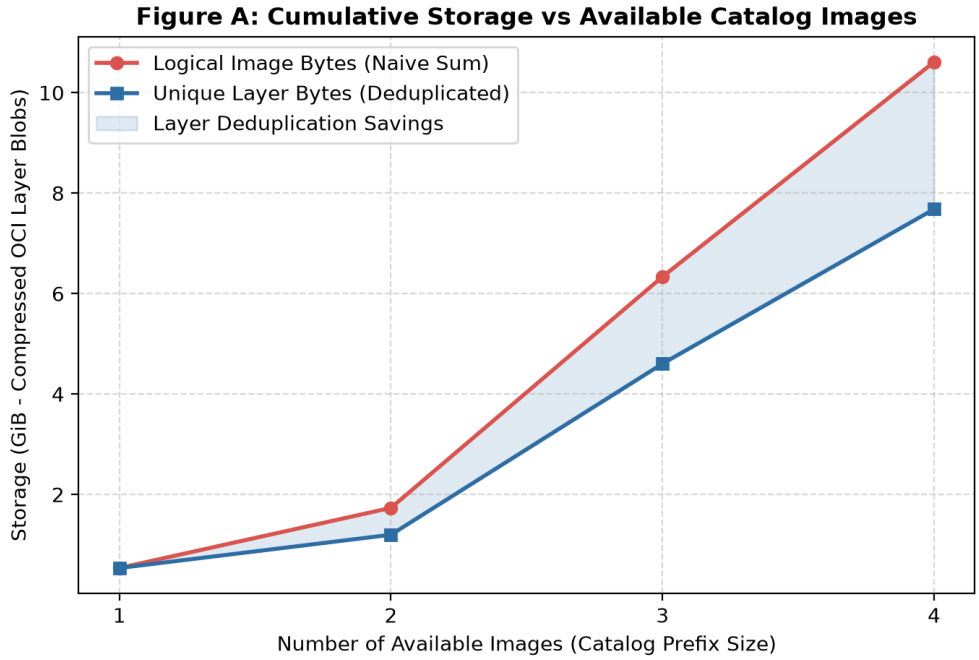


Figure 4.7. Cumulative compressed OCI layer storage across the four-image catalog prefix in Stream E5, demonstrating a 27.61% deduplication saving achieved through shared base layers.

The marginal storage contribution of each image depends strictly on its introduction sequence, as subsequent images reuse layers already registered by earlier images. Table 4.7 documents the marginal unique byte contributions in the evaluated sequence, and Figure 4.8 visualizes these marginal additions.

Table 4.7. Marginal unique compressed layer bytes introduced by image order

Introduction order	Container image identifier	Marginal unique bytes
1	`minimal-python`	574,631,877 (0.53 GiB)
2	`scipy-data-science`	710,803,665 (0.66 GiB)
3	`pytorch-deep-learning`	3,657,420,085 (3.41 GiB)
4	`tensorflow-deep-learning`	3,304,575,398 (3.08 GiB)

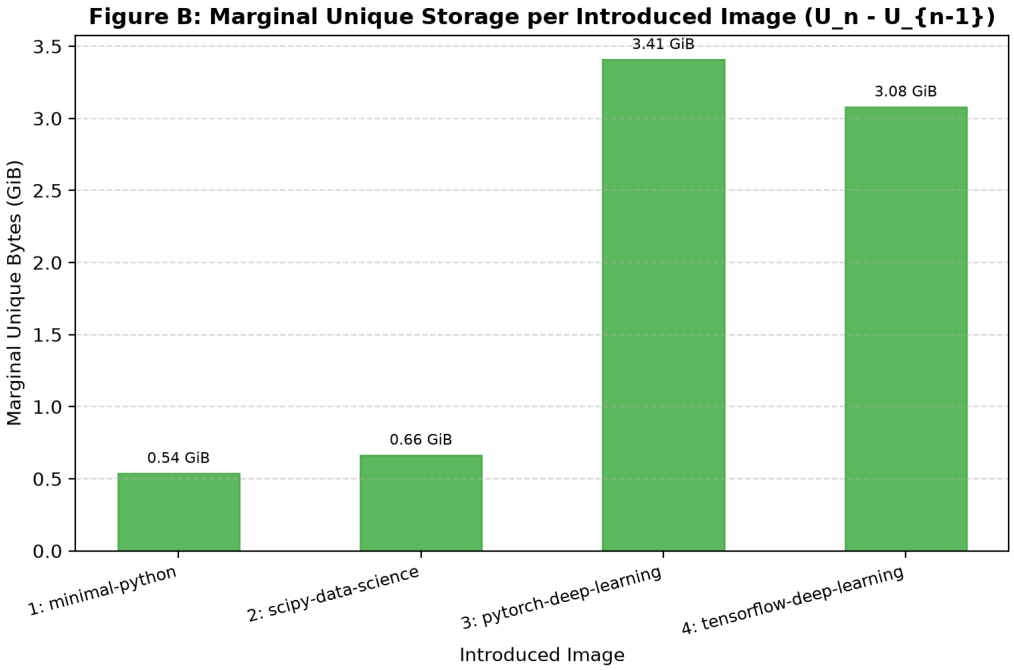


Figure 4.8. Marginal unique compressed layer storage introduced by each container image in Stream E5, showing shared base layer reuse between general-purpose and deep learning images.

The marginal analysis indicates that introducing `minimal-python` and `scipy-data-science` required only 0.53 GiB and 0.66 GiB of unique compressed storage, respectively. When deep learning images were subsequently added, `pytorch-deep-learning` introduced 3.41 GiB of unique layers, while `tensorflow-deep-learning` added 3.08 GiB.

To examine the specific structural overlap between image pairs, Figure 4.9 presents a pairwise shared storage heatmap, displaying the exact gigabytes of compressed layer blobs shared between each pair of images.

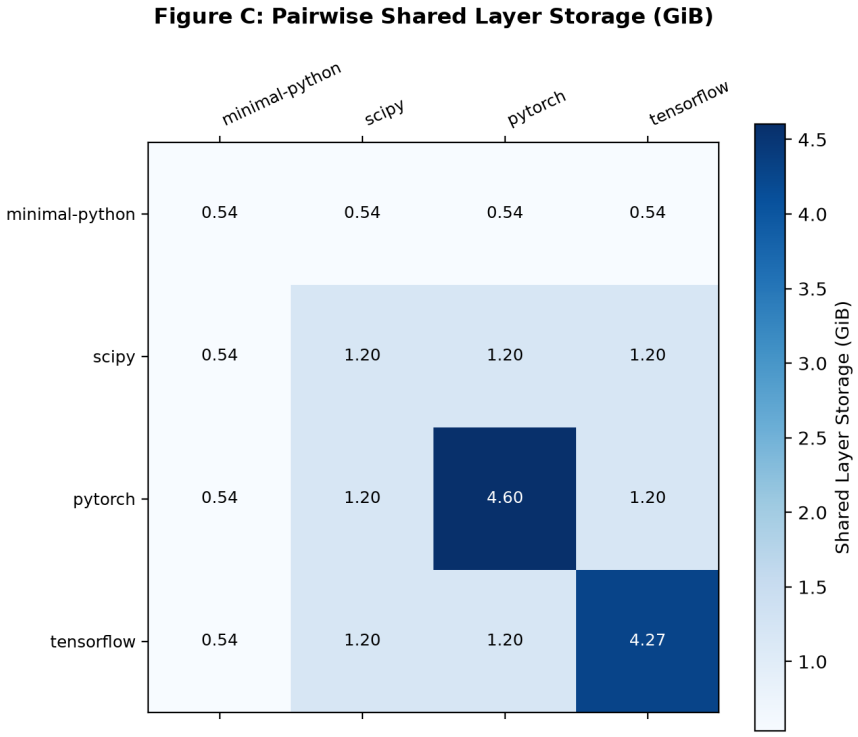


Figure 4.9. Pairwise shared layer storage heatmap (GiB) among the four approved container images, highlighting substantial layer inheritance between data science and deep learning environments.

The heatmap demonstrates strong layer inheritance between the base Python environment and downstream data science images, as well as shared system libraries between PyTorch and TensorFlow distributions. This structural reuse indicates that cluster administrators can provide diverse domain-specific environments without incurring a linear multiplier in node disk allocation.

4.5. User study design

4.5.1. Study protocol and task design

Stream E3 is designed to evaluate the human-computer interaction impact of intent-aware recommendation compared with manual environment selection. In standard JupyterHub deployments, users must inspect resource profile tables and image descriptions manually, a process that can introduce cognitive overhead and lead to suboptimal environment choices. Stream E3 formulates a rigorous, within-participant experimental

protocol to compare the manual baseline, B0, against the proposed intent-aware workflow, P2.

Under the experimental protocol, participants are assigned a series of representative interactive computing tasks, each characterized by specific workload requirements (such as model training, data analysis, or deep learning inference). To eliminate order effects and learning bias, the protocol implements a counterbalanced Latin square design, alternating the presentation order of B0 and P2 across participants. In both conditions, participants operate over the identical administrator-approved catalog, ensuring that P2 receives no private catalog information unavailable to B0.

The primary outcome metrics defined in the protocol include task completion time (seconds from task presentation to notebook launch), task correctness (proportion of selections satisfying workload constraints), decision confidence (measured via 5-point Likert scales), perceived cognitive workload (measured via the NASA-TLX instrument), and system usability (assessed through the System Usability Scale, SUS [**?**], alongside post-task single-question usability metrics [**?**]).

4.5.2. Limitations and scope

At the current stage of repository development, the E3 testing harness, user interface instrumentation, and experimental protocol have been implemented and verified via automated smoke tests. However, no human participant trials have been conducted.

In adherence to strict academic honesty, this report explicitly reports that Stream E3 contains zero participant observations. Consequently, no empirical claims are made regarding reductions in task completion time, improvements in user decision accuracy, decreases in cognitive workload, or superior subjective usability scores.

The absence of human experimental data is an essential distinction. High algorithmic accuracy in offline benchmarks does not automatically guarantee improved human decision-making. Users may misunderstand automated recommendations, override correct recommendations with personal preferences, or find manual selection simpler when

catalogs contain very few options. Conducting full human-subject trials under the verified protocol remains an essential objective for future work.

Chapter 5.

Conclusion

5.1. Conclusion

The internship project implemented an intent-aware profile and image recommendation workflow for JupyterHub. It extracts structured workload intent, retrieves catalog candidates through sparse and dense channels, applies deterministic hard constraints, and presents the result for user review and confirmation. The administrator-managed catalog and pre-spawn validation keep deployment choices within approved profiles and digest-pinned images.

E1 evaluated recommendation quality on 11 workload families with effective paired outcomes from the broader development benchmark. JointAccept@1 was 60.0% for P1, 65.0% for P2 local, and 55.9% for P2 LLM; the observed differences were not statistically significant. The retrieval ablation also showed that BM25 achieved higher Recall@1 (46.4%) than hybrid RRF (34.1%) and dense retrieval (24.3%) on this catalog. These results do not establish that the hybrid method is superior; they show a trade-off that should be assessed against the query and catalog distribution. E2 measured prompt robustness across 18 semantic families, with macro SRR of 56.8% for P1, 60.6% for P2 local, and 51.5% for P2 LLM.

E3 has a completed human-study protocol and testing harness but zero participant observations, so this report makes no empirical claim about user success, decision time, workload, or usability. In E4, 770 of 800 Pod trials succeeded (96.3%), with 30 OOM terminations and no timeouts across four allocation conditions in one OrbStack-backed cluster. E5 found that the four-image catalog covered 69.9% of cases for P1 and 84.3% for P2; all executed eligible probes passed (65/65 and 76/76). These observations support

the bounded implementation and the specific tests performed. They do not establish broad production performance, human effectiveness, or scalability to larger image catalogs.

5.2. Future work

The experimental results are subject to several limitations. The evaluation is based on a controlled resource and image catalog and therefore does not cover the full diversity of workloads, Kubernetes clusters, and software environments encountered in production. The human-study protocol has not yet been run with participants; future work should conduct the study and report results from the recruited participant population. In addition, image functionality depends on the specific images and probe requirements included in the evaluation. Future work can extend the evaluation to larger and more heterogeneous catalogs, additional workload domains, and longer-term user studies. Historical workload information could also be incorporated to improve recommendations based on previously observed resource usage. Further work on retrieval and ranking could investigate how recommendation quality changes as the catalog becomes larger and more diverse. Language-model-based re-ranking could also be explored as an optional additional stage after retrieval, provided that its computational cost and effect on recommendation quality are evaluated systematically. Finally, runtime resource measurements could be used to calibrate recommended resource allocations against actual workload behavior.

References

[1] Ali Anwar, Mohamed Mohamed, Vasily Tarasov, Michael Little, Lukas Rupprecht, Yue Cheng, Nannan Zhao, Dimitrios Skourtis, Amit S. Warke, He He, and Ali R. Butt. Improving Docker registry design based on production workload analysis. In *Proceedings of the 2018 USENIX Annual Technical Conference (USENIX ATC 18)*, pages 265–278, 2018.

[2] John Brooke. SUS: A 'quick and dirty' usability scale. In *Usability Evaluation in Industry*, pages 189–194. Taylor & Francis, 1996.

[3] Alexander Clemm, Laurent Ciavaglia, Leeyoung Lisheng, Kiran Makhijani, and Jeff Jefferts. Intent-based networking - concepts and definitions. RFC 9315, IETF, 2022.

[4] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 09)*, pages 758–759, 2009.

[5] Eli Cortez, Anand Bonde, Alexandre Muzio, Mark Russinovich, Yang Qi, and Ricardo Bianchini. Resource Central: Understanding and predicting workloads for improved resource management in large cloud platforms. In *Proceedings of the 26th ACM Symposium on Operating Systems Principles (SOSP 17)*, pages 153–167, 2017.

[6] Docker Documentation. About storage drivers. `https://docs.docker.com/engine/storage/drivers/`, 2026. Accessed: Sep. 2026.

[7] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[8] Alexander Felfernig, Gerhard Friedrich, Dietmar Jannach, and Markus Zanker. Constraint-based recommender systems. In *Recommender Systems Handbook*, pages 161–190. Springer, 2011.

[9] Tyler Harter, Brandon Salmon, Rose Liu, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Slacker: Fast distribution with lazy Docker containers. In *Proceedings of the 14th USENIX Conference on File and Storage Technologies (FAST 16)*, pages 181–195, 2016.

[10] Jonathan L. Herlocker, Joseph A. Konstan, Loren G. Terveen, and John T. Riedl. Evaluating collaborative filtering recommender systems. *ACM Transactions on Information Systems*, 22(1):5–53, 2004.

[11] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

[12] Anthony Jameson, Martijn C. Willemsen, Alexander Felfernig, Marco de Gemmis, Pasquale Lops, Giovanni Semeraro, and Li Chen. Human decision making and recommender systems. In *Recommender Systems Handbook*, pages 611–648. Springer, 2nd edition, 2015.

[13] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002.

[14] JupyterHub Contributors. KubeSpawner documentation. `https://jupyterhub-kubespawner.readthedocs.io/`, 2026. Accessed: Sep. 2026.

[15] JupyterHub Development Team. Jupyterhub: A conceptual overview. `https://jupyterhub.readthedocs.io/en/latest/explanation/concepts.html`, 2026. Accessed: Sep. 2026.

[16] JupyterHub Development Team. Spawners. `https://jupyterhub.readthedocs.io/en/stable/reference/spawners.html`, 2026. Accessed: Sep. 2026.

[17] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP 2020)*, pages 6769–6781, 2020.

[18] Thomas Kluyver, Benjamin Ragan-Kelley, Fernando Pérez, Brian Granger, Matthias

Bussonnier, Jonathan Frederic, Kyle Kelley, Jessica Hamrick, Jason Grout, Sylvain Corlay, Paul Ivanov, Damián Avila, Safia Abdalla, and Carol Willing. Jupyter notebooks—a publishing format for reproducible computational workflows. In *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, pages 87–90. IOS Press, 2016.

[19] Bart P. Knijnenburg, Martijn C. Willemsen, Zeno Gantner, Hakan Soncu, and Chris Newell. Explaining the user experience of recommender systems. *User Modeling and User-Adapted Interaction*, 22(4–5):441–504, 2012.

[20] Kubernetes Authors. Kubernetes scheduler. `https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/`, 2026. Accessed: Sep. 2026.

[21] Kubernetes Authors. Pod quality of service classes. `https://kubernetes.io/docs/concepts/workloads/pods/pod-qos/`, 2026. Accessed: Sep. 2026.

[22] Kubernetes Authors. Resource management for pods and containers. `https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/`, 2026. Accessed: Sep. 2026.

[23] Kubernetes Authors. Resource quotas. `https://kubernetes.io/docs/concepts/policy/resource-quotas/`, 2026. Accessed: Sep. 2026.

[24] Kubernetes Authors. Schedule GPUs. `https://kubernetes.io/docs/tasks/manage-gpus/scheduling-gpus/`, 2026. Accessed: Sep. 2026.

[25] Kubernetes Autoscaling SIG. Vertical pod autoscaler. `https://github.com/kubernetes/autoscaler/tree/master/vertical-pod-autoscaler`, 2026. Accessed: Sep. 2026.

[26] Aris Leivadeas and Matthias Falkner. A survey on intent-based networking. *IEEE Communications Surveys & Tutorials*, 25(1):625–655, 2023.

[27] Xi Victoria Lin, Chenglong Wang, Luke Zettlemoyer, and Michael D. Ernst. NL2Bash: A corpus and semantic clips parser for natural language interface to the Linux operating system. In *Proceedings of the 11th International Conference on*

Language Resources and Evaluation (LREC 18), pages 3502–3509, 2018.

[28] Yi Luan, Jacob Eisenstein, Kristina Toutanova, and Michael Collins. Sparse, dense, and attentional representations for text retrieval. *Transactions of the Association for Computational Linguistics*, 9:329–345, 2021.

[29] Open Container Initiative. OCI image format specification. `https://github.com/opencontainers/image-spec`, 2026. Accessed: Sep. 2026.

[30] Raja Parasuraman, Thomas B. Sheridan, and Christopher D. Wickens. A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics - Part A: Systems and Humans*, 30(3):286–297, 2000.

[31] Project Jupyter. JupyterHub documentation. `https://jupyterhub.readthedocs.io/`, 2026. Accessed: Sep. 2026.

[32] Pearl Pu, Li Chen, and Rong Hu. A user-centric evaluation framework for recommender systems. In *Proceedings of the 5th ACM Conference on Recommender Systems (RecSys 11)*, pages 157–164, 2011.

[33] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP 2019)*, pages 3982–3992, 2019.

[34] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[35] Michał Rządca, Paweł Findeisen, Jacek Świętek, Przemysław Zych, Przemysław Bialik, Marta Siedlecka, Jacek Wójcik, Dominik Krysiuk, Zuzanna Skowron, Ricardo Koller, and John Wilkes. Autopilot: Workload autoscaling at Google. In *Proceedings of the 15th European Conference on Computer Systems (EuroSys 20)*, pages 1–16, 2020.

[36] Jeff Sauro and Joseph S. Dumas. Comparison of three one-question, post-task

usability questionnaires. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI 09)*, pages 1599–1608, 2009.

[37] Qin Wu, Alexander Clemm, Mohamed Boucadair, Christian Jacquenet, and Chongfeng Xie. Intent-driven cloud resource design. In *Proceedings of the 2021 IEEE Cloud Summit*, pages 141–146, 2021.

[38] Zero to JupyterHub Contributors. Zero to JupyterHub with Kubernetes. `https://z2jh.jupyter.org/`, 2026. Accessed: Sep. 2026.